Go (Golang)

SDE2 Interview Guide

100 Questions & Answers with Code Examples

Level: Mid (2–4 yrs)	Stack: Go / Golang	Roles: SDE2 / Backend

> **Focus:** SQS Fundamentals, Standard vs FIFO, Visibility Timeout, DLQ, SNS+SQS, SQS vs Kafka, Lambda Triggers, Go AWS SDK | **Level:** SDE2

Table of Contents

SQS Fundamentals

Q1. What is Amazon SQS?

Difficulty: Easy

Amazon SQS (Simple Queue Service) is a fully managed message queuing service. It decouples producers and consumers, enabling asynchronous processing without managing queue infrastructure.

```
Producer → [SQS Queue] → Consumer(s)

Key properties:
  Fully managed: no servers to provision, patch, or scale
  Highly available: 3 AZ redundancy within a region
  Scalable: handles any message volume automatically
  At-least-once delivery: messages delivered at least once (Standard queue)
  Pull-based: consumers poll the queue (not push)
  Pay-per-use: charged per API call and data transfer

Message limits:
  Max message size: 256 KB
  Retention: 1 minute to 14 days (default 4 days)
  Max in-flight (Standard): 120,000
  Max in-flight (FIFO): 20,000

Pricing (us-east-1):
  First 1M requests/month: free
  Standard: $0.40 per million requests
  FIFO: $0.50 per million requests
```

Q2. What are the core SQS operations?

Difficulty: Easy

```
SendMessage:    publish message to queue
ReceiveMessage: poll queue for up to 10 messages
DeleteMessage:  acknowledge and remove message after processing
ChangeMessageVisibility: extend or reduce visibility timeout
GetQueueAttributes: queue metadata (depth, ARN, etc.)

Flow:
  1. Producer: SendMessage (message enters queue)
  2. Consumer: ReceiveMessage (message becomes invisible)
  3. Consumer: process message
  4. Consumer: DeleteMessage (remove permanently)
  OR
  4b: Visibility timeout expires → message becomes visible again → retry
```

```
Important: ReceiveMessage does NOT delete the message!
  You must explicitly call DeleteMessage after successful processing
  This enables at-least-once delivery (safe retry on failure)

Batch operations (up to 10 messages per call):
  SendMessageBatch:     send up to 10 messages in one API call
  DeleteMessageBatch:   delete up to 10 messages in one call
  ChangeMessageVisibilityBatch: change timeout for up to 10 messages
  → Reduces API calls, lowers cost, improves throughput
```

Q3. What is the SQS message structure?

Difficulty: Easy

```
SQS Message fields:
  MessageId:          unique identifier assigned by SQS
  ReceiptHandle:      token for DeleteMessage/ChangeVisibility (changes each receive)
  Body:               message content (string, max 256KB)
  MD5OfBody:          checksum for data integrity verification
  Attributes:         system attributes (ApproximateReceiveCount, SentTimestamp, etc.)
  MessageAttributes:  custom metadata (type-safe key-value pairs)

MessageAttributes (custom metadata):
  Name:     string key (max 256 chars)
  Type:     String | Number | Binary
  Value:    the data

// Send with attributes:
{
  "Body": "{\"orderId\": 42, \"amount\": 99.99}",
  "MessageAttributes": {
    "EventType": {
      "DataType": "String",
      "StringValue": "order.created"
    },
    "TraceId": {
      "DataType": "String",
      "StringValue": "abc-123-xyz"
    }
  }
}

// System attributes on received message:
ApproximateReceiveCount:          how many times received (retry count)
ApproximateFirstReceiveTimestamp: first time message was received
SentTimestamp:                    when message was sent
SenderId:                         IAM user/role that sent it
```

Q4. What is long polling vs short polling?

Difficulty: Medium

```
Short polling (WaitTimeSeconds=0, default):
  ReceiveMessage returns IMMEDIATELY
  Samples only a subset of servers
  May return empty even when messages exist
  Higher cost: many API calls return nothing

Long polling (WaitTimeSeconds=1-20):
  ReceiveMessage WAITS up to 20 seconds for messages to arrive
```

```
  Queries ALL servers (not just a subset)
  Returns empty only if no messages arrive during wait period
  Lower cost: fewer empty API calls
  Recommended for all production use

Enable long polling:
  Queue-level: QueueAttribute ReceiveMessageWaitTimeSeconds=20
  Per-request: WaitTimeSeconds=20 in ReceiveMessage call

Cost comparison example (100 messages/hour, empty otherwise):
  Short polling: 3600 × $0.40/M = minimal but wasteful
  Long polling: 300 × $0.40/M = 12× fewer API calls, same messages

Always use long polling (WaitTimeSeconds=20) in production:
  Reduces cost
  Reduces CPU on consumer
  Better latency (detects message arrival instantly vs polling delay)
```

Q5. What is the SQS visibility timeout?

Difficulty: Medium

```
Visibility timeout: period during which a received message is invisible
  After ReceiveMessage: message hidden from other consumers
  Consumer processes: if success → DeleteMessage (permanent removal)
  If timeout expires without DeleteMessage: message becomes visible again → retry

Default: 30 seconds
Range: 0 seconds to 12 hours
Queue-level default + per-message override possible

Timeline:
  t=0:  Message arrives in queue
  t=10: Consumer A receives message (invisible for 30s)
  t=25: Consumer A still processing...
  t=35: Timeout expires! Message visible again
  t=36: Consumer B receives the same message (duplicate!)
  t=40: Consumer A finishes and calls DeleteMessage (409 error - receipt handle invalid)
  t=41: Consumer B finishes and calls DeleteMessage (success)

Setting correctly:
  visibility_timeout >= max_processing_time × 6
  If processing takes 2 minutes: set to 12+ minutes

Extending timeout dynamically:
  Before timeout expires: call ChangeMessageVisibility
  Add more time when processing will take longer than expected
  Max: total visibility cannot exceed 12 hours
```

Q6. What are SQS queue attributes?

Difficulty: Medium

```
Key queue attributes:

VisibilityTimeout (default 30s, max 12h):
  How long a received message is invisible

MessageRetentionPeriod (default 4 days, 1min-14days):
```

```
  How long SQS keeps undeleted messages

MaximumMessageSize (default 256KB, 1KB-256KB):
  Max message body size

DelaySeconds (default 0, max 15min):
  Delivery delay for new messages (initially invisible)

ReceiveMessageWaitTimeSeconds (default 0, max 20s):
  Long polling wait time at queue level

RedrivePolicy:
  maxReceiveCount: failures before → DLQ
  deadLetterTargetArn: DLQ queue ARN

Policy:
  Resource-based policy (who can send/receive)

ApproximateNumberOfMessages:    messages in queue
ApproximateNumberOfMessagesNotVisible: in-flight (being processed)
ApproximateNumberOfMessagesDelayed:    delayed messages

Monitor these metrics in CloudWatch:
  NumberOfMessagesSent
  NumberOfMessagesDeleted
  NumberOfMessagesReceived
  ApproximateNumberOfMessagesVisible → alert if growing (consumers lagging)
  ApproximateAgeOfOldestMessage → alert if too high
```

Q7. What is message deduplication in SQS?

Difficulty: Medium

```
Standard queues: NO deduplication (at-least-once delivery)
  Duplicate messages possible (rare but can happen)
  Consumer MUST be idempotent

FIFO queues: deduplication within 5-minute window
  ContentBasedDeduplication: SQS computes SHA-256 of body → dedup key
  MessageDeduplicationId: custom dedup key (override)
  If same dedup ID received within 5 min → silently discarded

Content-based deduplication:
  Enable: ContentBasedDeduplication=true on FIFO queue
  SQS hashes message body → rejects same body within 5 minutes
  Limitation: same content = rejected even if genuinely different message
  Better: set unique MessageDeduplicationId per logical message

Custom deduplication ID:
  SendMessage with MessageDeduplicationId: "order-42-created-2024-01-15T10:30:00Z"
  Same ID in 5 min window → second message silently ignored
  First message already delivered → idempotent

Application-level idempotency (for Standard queues):
  Check if event already processed before doing work
  Store processed message IDs in DynamoDB/Redis with TTL
  Conditional DB update: only apply if state matches expected state
```

Q8. What is the SQS Extended Client Library?

Difficulty: Medium

```
Problem: SQS max message size = 256 KB
  Large payloads (images, large JSON, PDFs) don't fit

Solution: SQS Extended Client Library
  Stores large payload in S3
  Sends SQS message with S3 reference (pointer)
  Consumer: reads SQS message → fetches from S3 → processes

Flow:
  Producer:
    payload > 256KB → upload to S3 → send SQS message with S3 key
    payload ≤ 256KB → send directly in SQS message

  Consumer:
    Receive SQS message
    If S3 pointer: download from S3 → get full payload
    Process payload
    Delete SQS message + optionally delete S3 object

Message format (with S3 reference):
  [
    "software.amazon.payloadoffloading.PayloadS3Pointer",
    {"s3BucketName": "my-bucket", "s3Key": "messages/uuid"}
  ]

Manual implementation in Go (without official library):
  const maxSQSSize = 256 * 1024
  if len(payload) > maxSQSSize {
      key := uploadToS3(payload)
      sqsBody = `{"s3_key":"` + key + `"}`
  } else {
      sqsBody = string(payload)
  }
```

Q9. What is the ApproximateNumberOfMessages metric?

Difficulty: Easy

```
CloudWatch metrics for SQS monitoring:

ApproximateNumberOfMessagesVisible:
  Messages available for retrieval (queue depth)
  Growing → consumers can't keep up → scale consumers
  Alert: > threshold for > 5 minutes

ApproximateNumberOfMessagesNotVisible:
  In-flight: received but not yet deleted
  Max: 120,000 (Standard), 20,000 (FIFO)
  High value → slow processing or processing hanging

ApproximateNumberOfMessagesDelayed:
  Waiting due to delay policy
  Expected for delayed queues, alert if unexpected

ApproximateAgeOfOldestMessage:
  Seconds since oldest unprocessed message was sent
```

```
  High → messages building up (backlog)
  Alert: > retention_period × 0.8 (risk of message expiry)

NumberOfMessagesSent:       producer throughput
NumberOfMessagesDeleted:    consumer throughput
NumberOfMessagesReceived:   receive API calls

Auto-scaling with KEDA (Kubernetes):
  Scale consumer pods based on ApproximateNumberOfMessagesVisible
  lagThreshold: 100  → target 1 message per pod per second

Auto-scaling with AWS Application Auto Scaling:
  Scaling policy: scale ECS tasks when queue depth > 100
```

Q10. What are SQS message timers (delay queues)?

Difficulty: Medium

```
Delay Queues: messages initially invisible for N seconds after send
  All messages delayed → use Queue DelaySeconds attribute
  Queue-level: 0-900 seconds (15 minutes) default delay

Per-message delay:
  SendMessage with DelaySeconds parameter
  Individual message delayed, not entire queue
  Max: 900 seconds (15 minutes)
  Standard queues: override queue default
  FIFO queues: not supported (use queue-level only)

Use cases:
  Retry with backoff: delay retried messages
  Scheduled tasks: "send email 5 min after registration"
  Webhook retries: delay before retrying failed webhook
  Order pipeline: "charge card 10 min after order placed" (fraud window)

Limitation: max 15 minutes
  For longer delays: use EventBridge Scheduler or Step Functions Wait state

// Go: send with 5 minute delay
svc.SendMessage(&sqs.SendMessageInput{
    QueueUrl:     &queueURL,
    MessageBody:  aws.String(body),
    DelaySeconds: aws.Int64(300),  // 5 minutes
})

Pattern: exponential backoff retries via delay
  Attempt 1: delay 0s
  Attempt 2: delay 30s
  Attempt 3: delay 120s
  Attempt 4: delay 300s (5min max for SQS)
  Attempt 5+: → Dead Letter Queue
```

Q11. What is the SQS message receipt handle?

Difficulty: Medium

```
ReceiptHandle: token returned with ReceiveMessage
  Required for: DeleteMessage, ChangeMessageVisibility
  NOT the MessageId (different field!)
```

```
  Changes each time message is received (new receipt handle)

Why it changes:
  Prevents "stale" operations from old consumers
  Consumer A receives (handle-A), times out
  Consumer B receives (handle-B)
  Consumer A tries DeleteMessage with handle-A → error (stale handle)
  Consumer B deletes with handle-B → success

ReceiptHandle is 1000+ characters (internal SQS metadata)

// Go: store receipt handle from receive, use for delete
result, _ := svc.ReceiveMessage(&sqs.ReceiveMessageInput{...})
for _, msg := range result.Messages {
    handle := msg.ReceiptHandle  // save this!

    if err := process(*msg.Body); err == nil {
        svc.DeleteMessage(&sqs.DeleteMessageInput{
            QueueUrl:      &queueURL,
            ReceiptHandle: handle,  // use saved handle
        })
    }
}

Important: receipt handle is per-consumer, per-receive
  Multiple consumers: each gets different handle for same message
  Don't share receipt handles across goroutines
```

Q12. What is SQS access control?

Difficulty: Medium

```
Two mechanisms for SQS access control:

1. IAM Policies (identity-based):
   Attach to IAM user/role/group
   {
     "Effect": "Allow",
     "Action": ["sqs:SendMessage", "sqs:ReceiveMessage", "sqs:DeleteMessage"],
     "Resource": "arn:aws:sqs:us-east-1:123456789:my-queue"
   }

2. Queue Policy (resource-based):
   Attached to the queue itself
   Controls who can access this queue
   Required for: cross-account access, SNS publish

Example: allow SNS to publish to SQS queue:
  {
    "Effect": "Allow",
    "Principal": {"Service": "sns.amazonaws.com"},
    "Action": "sqs:SendMessage",
    "Resource": "arn:aws:sqs:...:my-queue",
    "Condition": {
      "ArnEquals": {"aws:SourceArn": "arn:aws:sns:...:my-topic"}
    }
  }

VPC Endpoint for SQS:
```

```
  Access SQS without internet (private network)
  com.amazonaws.region.sqs endpoint
  Policy restricts to specific queues

Least privilege:
  Producer: sqs:SendMessage only
  Consumer: sqs:ReceiveMessage + sqs:DeleteMessage + sqs:ChangeMessageVisibility
  Admin: sqs:* (only for management tooling)
```

Q13. What is SQS server-side encryption? Difficulty: Easy

```
SQS SSE: encrypt messages at rest using KMS
  All messages encrypted before storing
  Decrypted automatically on ReceiveMessage
  Transparent to application code

KMS key options:
  AWS managed key (aws/sqs): free, simpler
  Customer managed key (CMK): control over key, rotation, audit

Enable encryption:
  Queue attribute: SqsManagedSseEnabled=true (AWS managed)
  Or: KmsMasterKeyId=<key-id> (CMK)

Encryption scope:
  At-rest: stored in SQS infrastructure
  In-transit: always TLS (HTTPS endpoints)

KMS key policy required:
  Key policy must allow SQS to use it

Performance:
  KMS calls: one call per unique (queue, key) pair cached for 5 minutes
  Not one KMS call per message (cached)
  High-throughput: minimal KMS latency impact

Compliance use cases:
  PCI DSS: encrypt cardholder data in transit through SQS
  HIPAA: encrypt PHI in message queues
  GDPR: encrypt personal data at rest
```

Q14. What are SQS limits you must know? Difficulty: Easy

```
Message limits:
  Max size:          256 KB (use S3 Extended Client for larger)
  Retention:         1 min to 14 days (default 4 days)
  Delay:             0 to 900 seconds (15 minutes)
  Visibility timeout: 0 to 12 hours (default 30 seconds)

Throughput limits:
  Standard queue:    unlimited messages/second
  FIFO queue:        300 msg/sec (basic)
                     3,000 msg/sec (with batching, 10 msg/batch)

In-flight limits:
  Standard: 120,000 messages in-flight simultaneously
```

```
  FIFO:     20,000 messages in-flight simultaneously

Batch limits:
  SendMessageBatch: 10 messages, total 256 KB
  DeleteMessageBatch: 10 messages
  ReceiveMessage: up to 10 messages per call

Per-account limits:
  Default: 1,000 queues per region per account (can request increase)
  Active message quota: up to 120,000 inflight Standard

Long polling:
  Max WaitTimeSeconds: 20 seconds

Message deduplication (FIFO):
  5-minute deduplication window
```

Q15. ### Q15-Q20: More SQS Fundamentals — Difficulty: Medium

Q	Topic
Q15	SQS pricing model and cost optimization strategies
Q16	SQS message filtering vs routing
Q17	SQS purge queue: when and how to use it
Q18	SQS queue naming conventions and tagging
Q19	SQS monitoring with CloudWatch dashboards
Q20	SQS vs SNS vs EventBridge: which to use

Standard vs FIFO Queues

Q16. What is the difference between Standard and FIFO queues? — Difficulty: Medium

```
                Standard            FIFO
Ordering        Best-effort         Strict (FIFO within group)
Delivery        At-least-once       Exactly-once
Deduplication   None                5-min window
Throughput      Unlimited           300/sec (3000 with batching)
Cost            $0.40/M            $0.50/M (25% more)
Use case        High throughput     Ordering + no duplicates
Name suffix     any name            must end with .fifo

Standard queue:
  May deliver out of order (rare but possible)
  May deliver same message more than once (consumer must be idempotent)
  Unlimited throughput: burst millions of messages
  Most use cases

FIFO queue:
```

```
  Strict ordering: first sent = first received (within message group)
  Exactly-once: deduplication by MessageDeduplicationId
  Lower throughput: 300 TPS (3,000 TPS with batching)
  Required: financial transactions, ordered event streams, idempotent workflows

Decision rule:
  "Do I need strict ordering OR guaranteed no duplicates?"
  YES → FIFO
  NO  → Standard (simpler, cheaper, higher throughput)
```

Q17. What is a FIFO queue message group?

Difficulty: Hard

```
MessageGroupId: logical partition within FIFO queue
  Messages with same GroupId: strictly ordered (FIFO)
  Messages with different GroupId: can be processed in parallel

Without MessageGroupId:
  All messages in one implicit group → sequential processing
  Only 1 consumer active at a time → low throughput

With MessageGroupId:
  N groups → N consumers can process in parallel
  Ordering preserved per group, not globally

Example: order processing
  MessageGroupId = customer_id
  All orders for customer-42 are processed in order
  Orders for customer-99 processed simultaneously by another consumer

  Customer 42: create → add_item → checkout (in order)
  Customer 99: create → checkout (parallel with customer 42's orders)

Partitioning strategy:
  High cardinality: each entity gets own group → max parallelism
  Low cardinality: few groups → serialized within group → less parallelism

  Too few groups: bottleneck
  Too many groups: fine, just overhead of group tracking

FIFO In-flight limit:
  20,000 messages in-flight per queue
  Per group: unlimited in-flight (but ordered delivery within group)
```

Q18. When should you use FIFO over Standard?

Difficulty: Medium

```
Use FIFO when:

1. Order matters absolutely:
   Bank account transactions: deposit before withdrawal
   Order state machine: created → paid → shipped → delivered
   Event sourcing: events must be applied in sequence

2. Exactly-once processing required:
   Payment processing: charge card exactly once per order
   Inventory decrement: can't decrement twice for same order
   Email sending: don't send duplicate confirmation emails
```

```
3. Idempotency is hard to implement:
   Some operations are inherently non-idempotent
   FIFO's deduplication handles it at infrastructure level

Use Standard when:

1. High throughput needed (>3000 TPS):
   Log ingestion, metrics collection, analytics events

2. Order doesn't matter:
   Email notifications (order doesn't affect outcome)
   Image processing (each image independent)
   Background jobs (unrelated tasks)

3. Consumer is naturally idempotent:
   "Set user status to active" → safe to run twice

4. Cost-sensitive:
   Standard is 20% cheaper

Rule of thumb:
  Most backend jobs → Standard
  Financial/transactional workflows → FIFO
  Event sourcing → FIFO
  Parallel processing of independent tasks → Standard
```

Q19. What are FIFO queue best practices?

Difficulty: Medium

```
1. Set MessageDeduplicationId explicitly (don't rely on content-based):
   UUID per logical operation: "order-42-created-" + uuid.New().String()
   Predictable deduplication: same logical event = same dedup ID
   Content-based dedup: risk of false positives (same body, different event)

2. Choose MessageGroupId cardinality carefully:
   Per-entity (user/order ID): good parallelism, ordered per entity
   Global (all messages same group): sequential → use only if truly needed

3. Handle FIFO throughput limits:
   3,000 TPS with batching (10 msg/batch)
   If more needed: multiple FIFO queues + routing layer
   Or: reconsider if Standard queue fits (most cases it does)

4. Consumer concurrency for FIFO:
   One consumer per active message group (no parallel within group)
   N consumers for N groups = N parallel streams
   Don't consume same group ID in parallel (breaks ordering)

5. Monitoring FIFO-specific:
   NumberOfMessageGroupsWithInflightMessages
   NumberOfDeduplicatedMessages (are duplicates being filtered?)

6. FIFO queue naming:
   Must end with .fifo: my-orders-queue.fifo
   Important: include environment: orders-prod.fifo

// Go: send to FIFO queue
```

```
svc.SendMessage(&sqs.SendMessageInput{
    QueueUrl:               &queueURL,
    MessageBody:            aws.String(body),
    MessageGroupId:         aws.String(customerID),
    MessageDeduplicationId: aws.String(orderID + "-created"),
})
```

Q20. ### Q25-Q35: More Standard vs FIFO Topics

Difficulty: Medium

Q	Topic
Q25	FIFO queue consumer group design patterns
Q26	Standard queue ordering: why it's not guaranteed
Q27	Migrating from Standard to FIFO queue
Q28	FIFO high-throughput mode (3000 TPS) configuration
Q29	SQS FIFO with Lambda: concurrency considerations
Q30	Duplicate handling strategies for Standard queues
Q31	FIFO deduplication window: what happens after 5 minutes
Q32	SQS FIFO vs Kinesis: ordering comparison
Q33	Multiple FIFO queues for higher throughput patterns
Q34	FIFO queue with DLQ configuration
Q35	Exactly-once semantics: SQS FIFO limitations

Visibility Timeout & Message Lifecycle

Q21. How do you set the right visibility timeout?

Difficulty: Medium

```
Rule: visibility_timeout >= max_processing_time × 6
  Multiplier provides buffer for:
  - Slow processing (occasional)
  - Network latency
  - Retry overhead

Examples:
  Processing takes ~10s → timeout = 60s
  Processing takes ~1min → timeout = 6 minutes
  Processing takes ~5min → timeout = 30 minutes
  Processing takes ~1h → timeout = 6 hours (close to max 12h)

Too short:
  Message redelivered while still being processed
  Duplicate processing
  Extra load on consumers (processing same message multiple times)
```

```
Too long:
  Failure detection delayed (consumer crashes → message invisible for hours)
  Queue appears empty but has in-flight messages

Dynamic visibility timeout (best practice):
  Extend before expiry if still processing
  Heartbeat pattern: goroutine extends timeout every 30s during processing

  func processWithHeartbeat(msg *sqs.Message) error {
      done := make(chan struct{})
      go func() {  // heartbeat goroutine
          ticker := time.NewTicker(30 * time.Second)
          defer ticker.Stop()
          for {
              select {
              case <-done: return
              case <-ticker.C:
                  svc.ChangeMessageVisibility(&sqs.ChangeMessageVisibilityInput{
                      QueueUrl:          &queueURL,
                      ReceiptHandle:     msg.ReceiptHandle,
                      VisibilityTimeout: aws.Int64(60),
                  })
              }
          }
      }()
      defer close(done)
      return doWork(msg)
  }
```

Q22. What is the message lifecycle in SQS?

Difficulty: Medium

```
Message lifecycle states:

Available (visible):
  Message sits in queue waiting for consumer
  ReceiveMessage will return this message

In-flight (invisible):
  Message received but not deleted
  Other consumers cannot receive it
  Visibility timeout counting down

Delayed:
  Message not yet available due to delay policy
  Timer running; becomes Available when timer expires

Dead (in DLQ):
  After maxReceiveCount failures → moved to DLQ
  Stays in DLQ until manually processed or expired

Deleted:
  Consumer called DeleteMessage
  Message permanently removed from queue

Timing:
  t=0:   SendMessage → Available
```

```
  t=5:   ReceiveMessage → In-flight (invisible for timeout period)
  t=35:  VisibilityTimeout=30 expires without DeleteMessage → Available again
  t=36:  Different consumer receives → In-flight
  t=40:  DeleteMessage → Deleted (permanently gone)

If message received maxReceiveCount times without delete → DLQ
  ApproximateReceiveCount attribute tracks retry count
```

Q23. What happens when a consumer crashes?

Difficulty: Medium

```
Consumer crashes during processing:

Scenario:
  t=0:  Consumer receives message (visibility=60s)
  t=10: Consumer crashes (process killed, OOM, network loss)
  t=60: VisibilityTimeout expires → message becomes visible again
  t=61: Another consumer picks up message
  t=75: Second consumer processes successfully → DeleteMessage

Result: at-least-once delivery
  Message processed at least once (by the second consumer)
  If first consumer had partially applied changes: need idempotency

Without DeleteMessage: safe to retry
  If consumer crashes after work but before DeleteMessage:
  → Message retried → must be idempotent

Consumer restart strategy:
  On startup: check in-progress work table → clean up partial work
  Then: start consuming from SQS

Kubernetes pod crash:
  Pod restarts automatically (restartPolicy: Always)
  SQS message timeout ensures retry even if pod takes time to restart
  Set visibility_timeout > pod_restart_time + processing_time

Monitoring crashes:
  ApproximateReceiveCount > 1 → message being retried
  Alert when specific messages hit maxReceiveCount → DLQ
  CloudWatch: NumberOfMessagesReceived without corresponding DeleteMessage
```

Q24. What is ChangeMessageVisibility?

Difficulty: Medium

```
ChangeMessageVisibility: modify visibility timeout for an in-flight message
  Extend: give more time to process large/complex message
  Reduce (set to 0): make message immediately visible (abandon processing)

Parameters:
  ReceiptHandle: token from ReceiveMessage
  VisibilityTimeout: new timeout (0 to 12 hours)
  Setting to 0: returns message to queue immediately

Use cases:

Extend (more time needed):
```

```
  svc.ChangeMessageVisibility(&sqs.ChangeMessageVisibilityInput{
      QueueUrl:          &queueURL,
      ReceiptHandle:     msg.ReceiptHandle,
      VisibilityTimeout: aws.Int64(300),  // 5 more minutes
  })

Abandon (unrecoverable error, don't wait for timeout):
  svc.ChangeMessageVisibility(&sqs.ChangeMessageVisibilityInput{
      QueueUrl:          &queueURL,
      ReceiptHandle:     msg.ReceiptHandle,
      VisibilityTimeout: aws.Int64(0),  // return to queue now
  })

Heartbeat pattern (recommended for long-running tasks):
  Every 30s: extend timeout by 60s
  Ensures timeout doesn't expire during legitimate processing
  Stop heartbeat when done → delete or abandon

Note: cannot reduce timeout to < time already elapsed
  If 10s of original 30s remain: can extend, cannot set to < 0
```

Q25. ### Q40-Q50: More Visibility Timeout Topics — Difficulty: Medium

Q	Topic
Q40	Batch ChangeMessageVisibility for multiple messages
Q41	Visibility timeout vs message retention period
Q42	In-flight message limit: what happens when reached
Q43	Consumer scaling and visibility timeout interaction
Q44	Optimizing visibility timeout for different workload types
Q45	Message age monitoring: ApproximateAgeOfOldestMessage
Q46	Poison message detection via ApproximateReceiveCount
Q47	Message timer vs queue delay: which to use
Q48	Concurrent consumers and visibility timeout coordination
Q49	SQS backoff strategy with visibility timeout
Q50	Consumer group patterns for SQS

Dead Letter Queues & Error Handling

Q26. What is a Dead Letter Queue (DLQ)?

Difficulty: Medium

```
DLQ: queue for messages that fail processing N times
  After maxReceiveCount failures → automatically moved to DLQ
  Main queue: processed normally; DLQ: problem messages

Configuration:
  RedrivePolicy on main queue:
  {
    "maxReceiveCount": 5,        // failures before DLQ
    "deadLetterTargetArn": "arn:aws:sqs:...:orders-dlq"
  }

DLQ requirements:
  Same type as source (Standard DLQ for Standard, FIFO DLQ for FIFO)
  Same region and account as source queue
  DLQ should have LONGER retention than source queue

Why messages end up in DLQ:
  Processing exceptions (bug, invalid data)
  Downstream service down (DB unavailable)
  Malformed message (schema changed)
  Timeout exceeded (slow processing)
  Business rule violations

DLQ best practices:
  Always configure DLQ on every queue (don't let messages expire silently)
  DLQ retention = 14 days (max) → time to investigate
  Alert on DLQ depth > 0 (any message in DLQ = bug/issue)
  Process DLQ: fix bug → redrive (replay) to main queue

Monitoring:
  CloudWatch alarm: ApproximateNumberOfMessagesVisible > 0 on DLQ → SNS alert
```

Q27. How do you redrive messages from DLQ?

Difficulty: Medium

```
Redrive: replay DLQ messages back to the source queue
  After fixing the bug that caused failures

Methods:

1. AWS Console: "Start DLQ redrive" button
   Redrive all or subset of messages
   Rate: up to 500 messages/second

2. SQS Redrive API (AWS SDK):
   StartMessageMoveTask: move messages from DLQ to source
   ListMessageMoveTasks: check redrive progress
   CancelMessageMoveTask: stop if needed

3. Manual redrive (custom code):
   Poll DLQ → process differently (fix applied) → delete from DLQ
   More control: can filter which messages to redrive

// Go: manual DLQ redrive
```

```
for {
    result, _ := svc.ReceiveMessage(&sqs.ReceiveMessageInput{
        QueueUrl:            &dlqURL,
        MaxNumberOfMessages: aws.Int64(10),
        WaitTimeSeconds:     aws.Int64(20),
    })
    for _, msg := range result.Messages {
        // Re-send to main queue with fix applied
        svc.SendMessage(&sqs.SendMessageInput{
            QueueUrl:    &mainQueueURL,
            MessageBody: msg.Body,
        })
        svc.DeleteMessage(&sqs.DeleteMessageInput{
            QueueUrl:      &dlqURL,
            ReceiptHandle: msg.ReceiptHandle,
        })
    }
    if len(result.Messages) == 0 { break }
}

Redrive strategy:
  Don't just blindly redrive → messages will fail again!
  Fix the root cause first → verify fix → then redrive
```

Q28. What is the maxReceiveCount and how do you set it?

Difficulty: Medium

```
maxReceiveCount: how many times a message can be received before going to DLQ
  Counts: how many times ReceiveMessage returned this message
  After count exceeded: moved to DLQ automatically

Setting the right value:
  Transient failures (DB restart, network blip):
    maxReceiveCount = 3-5 (allow a few retries)

  External API rate limits (429 errors):
    maxReceiveCount = 10 + use delay between retries

  Never-succeed cases (malformed messages):
    maxReceiveCount = 3 (fail fast → DLQ quickly)

Interaction with visibility timeout:
  Each ReceiveMessage (and subsequent timeout) = +1 to ApproximateReceiveCount
  If consumer receives message but crashes before delete:
    Timeout expires → message visible → next receive = count+1

  maxReceiveCount = 5, visibility = 60s:
    Message retried 5 times over 5+ minutes → DLQ

// Check receive count in your consumer:
for _, msg := range result.Messages {
    receiveCount, _ := strconv.Atoi(
        aws.StringValue(msg.Attributes["ApproximateReceiveCount"]),
    )
    if receiveCount > 3 {
        // Close to DLQ, log extra details for debugging
        log.Warnf("message %s received %d times", *msg.MessageId, receiveCount)
```

```
    }
    processMessage(msg)
}
```

Q29. How do you handle different types of failures?

Difficulty: Hard

```
Failure categories and handling:

1. Transient failures (retry-able):
   Network timeout, DB connection failure, temporary unavailability
   → Don't delete, let visibility timeout expire → auto-retry
   → Or: ChangeMessageVisibility(0) to retry immediately
   → maxReceiveCount = 5

2. Business logic errors (non-retryable):
   Validation failure, missing required data, invariant violation
   → Move to DLQ immediately (don't waste retries)
   → Or: send to separate error queue for human review

   if isValidationError(err) {
       moveToErrorQueue(msg)
       svc.DeleteMessage(...)  // remove from main queue
       return
   }

3. Poison pills (malformed messages):
   Can never be processed regardless of retries
   → maxReceiveCount = 1 or 2 → fast path to DLQ
   → DLQ message with original payload for debugging

4. External API rate limits:
   429 Too Many Requests
   → ChangeMessageVisibility to delay retry
   → Increase visibility timeout to implement backoff

5. Idempotency violations:
   Message already processed (ApproximateReceiveCount > 1)
   Check idempotency key → if already done → delete + continue
   No failure, just duplicate detection

// Categorized error handling:
func handleSQSMessage(msg *sqs.Message) {
    err := processMessage(msg)
    if err == nil { deleteMessage(msg); return }

    if isNonRetryable(err) {
        sendToErrorQueue(msg, err)
        deleteMessage(msg)
    }
    // Otherwise: don't delete → let SQS retry (visibility timeout)
}
```

Q30. ### Q55-Q65: More DLQ Topics

Difficulty: Medium

Q	Topic

Q55	DLQ alarm configuration in CloudWatch
Q56	Multiple DLQs: per-error-type routing
Q57	DLQ message metadata for debugging (original timestamp, error)
Q58	DLQ retention period best practices
Q59	Redrive policy limits and considerations
Q60	DLQ with Lambda: partial batch failure handling
Q61	Circuit breaker pattern with SQS and DLQ
Q62	Error enrichment: adding context before DLQ
Q63	DLQ consumer for alerts and automated recovery
Q64	Message expiry: what happens when retention exceeded
Q65	Testing DLQ behavior in development

SNS + SQS Fan-Out Patterns

Q31. What is the SNS + SQS fan-out pattern?

Difficulty: Medium

```
Fan-out: one event → multiple independent consumers
  SNS Topic: broadcast to all subscribers
  SQS Queues: one per consumer service (buffers for each)

Architecture:
  OrderService → SNS:order-created → [SQS:email-queue, SQS:inventory-queue, SQS:analytics-queue]

  EmailService → polls SQS:email-queue → sends confirmation email
  InventoryService → polls SQS:inventory-queue → reserves stock
  AnalyticsService → polls SQS:analytics-queue → updates dashboards

Benefits:
  Decoupled: services don't know about each other
  Independent scaling: each service scales based on its own queue depth
  Independent failures: email service down → inventory unaffected
  Independent retry: each service retries independently
  Add new subscriber: create new SQS queue + SNS subscription (no code change)

vs Direct SNS → Lambda:
  With SQS buffer: handles bursts (SQS absorbs spike, Lambda processes at own rate)
  Without SQS: Lambda throttled if SNS bursts faster than Lambda concurrency

Setup:
  1. Create SNS topic
  2. Create SQS queue per consumer
  3. Subscribe each SQS queue to SNS topic
  4. SQS queue policy: allow SNS to send
  5. SNS subscription: filter policy (optional, per-service filtering)
```

Q32. What is SNS message filtering?

Difficulty: Medium

```
Subscription Filter Policy: each SQS subscriber receives only matching messages

Without filtering: every subscriber receives every SNS message
With filtering: subscribers receive only relevant messages

Filter policy (JSON on the subscription):
  {
    "eventType": ["order.created", "order.cancelled"],
    "region": ["us-east-1", "eu-west-1"]
  }

Message attributes must match filter for delivery:
  SNS message: {MessageAttributes: {eventType: "order.shipped"}}
  Filter: eventType: ["order.created"] → NOT delivered to this subscriber
  Filter: eventType: ["order.created", "order.shipped"] → DELIVERED

Filter operators:
  Exact match:  {"eventType": ["order.created"]}
  Prefix:       {"eventType": [{"prefix": "order."}]}
  Anything-but: {"eventType": [{"anything-but": ["payment.failed"]}]}
  Numeric:      {"amount": [{"numeric": [">=", 100]}]}
  Exists:       {"eventType": [{"exists": true}]}

Use cases:
  EmailService: subscribe to order.created only
  FraudService: subscribe to payment.* events
  HighValueAlerts: subscribe to orders with amount >= $1000
  RegionalService: subscribe to events for specific region

Benefits:
  Reduced queue depth (services only process relevant events)
  Lower cost (fewer messages delivered to non-interested services)
  Simplified consumer logic (no need to filter in code)
```

Q33. What is EventBridge vs SNS+SQS?

Difficulty: Hard

```
               SNS+SQS                  EventBridge
Routing        Topic-based (all subs)   Content-based (rules)
Filtering      Attribute filtering       Rich JSON pattern matching
Sources        Your apps + AWS services  100+ AWS services natively
Targets        SQS, Lambda, HTTP, email  20+ targets (SQS, Lambda, Step Functions, ...)
Schema         No schema enforcement     Schema registry available
Replay         No                       Event archive + replay
Cost           Per message              Per event + rules
Setup          Lower                    Higher (more powerful)

EventBridge advantages:
  Richer event routing (any JSON field, nested paths)
  Native AWS service events (EC2 start, S3 put, CodePipeline, etc.)
  Event archive + replay for debugging/reprocessing
  Schema registry for event discovery
  Cross-account event routing

SNS+SQS advantages:
```

```
  Simpler for basic fan-out
  Lower latency (SNS → SQS is faster than EventBridge)
  Message buffering per subscriber (SQS)
  Lower cost for simple routing

Decision:
  EventBridge: complex routing, AWS service events, event sourcing, replay
  SNS+SQS: simple fan-out to services, buffered delivery, basic filtering
  Both: can coexist (EventBridge → SNS → SQS for complex + buffered)
```

Q34. ### Q69-Q75: More SNS+SQS Topics

Difficulty: Medium

Q	Topic
Q69	SNS FIFO topic + SQS FIFO queue for ordered fan-out
Q70	Cross-account SNS → SQS: IAM and queue policy setup
Q71	SNS message structure and how it wraps SQS messages
Q72	HTTP/HTTPS SNS subscription with signature verification
Q73	SNS delivery retry policy and DLQ for SNS
Q74	SQS + EventBridge Pipes for transformation
Q75	Outbox pattern: DB → SQS vs DB → SNS → SQS

SQS vs Kafka & Architecture Decisions

Q35. When do you choose SQS over Kafka?

Difficulty: Medium

```
Choose SQS when:

1. Simplicity + managed service:
   No Kafka cluster to manage, patch, or scale
   AWS handles all operations

2. Simple task queue pattern:
   Produce task → single consumer processes it → delete
   No need for replay or multiple consumers

3. AWS ecosystem integration:
   Lambda triggers, EventBridge, SNS — native integration
   IAM-based auth (no Kafka credential management)

4. Variable/unpredictable workload:
   SQS scales to zero (no idle broker cost)
   Pay-per-message (not per-hour for cluster)

5. Small/medium volume (< 1M msg/day):
```

```
   SQS cost: nearly free at low volume
   Kafka on EC2: $50-200/month minimum

Choose Kafka when:

1. Message replay needed:
   Audit, reprocessing, event sourcing
   SQS: messages deleted after consumption

2. Multiple independent consumers:
   Both OrderService AND AnalyticsService read same event
   SQS: needs separate queue per consumer (duplication)
   Kafka: multiple consumer groups, one topic

3. High throughput (>100K msg/sec):
   SQS Standard: ~100K TPS per queue (works)
   But Kafka: lower cost and better performance at this scale

4. Strict ordering with high throughput:
   SQS FIFO: 3,000 TPS max
   Kafka: millions/sec with ordering per partition

5. Stream processing:
   Kafka Streams, Flink, Spark Streaming native Kafka integration
   SQS: no native stream processing
```

Q36. What is the SQS vs RabbitMQ comparison?

Difficulty: Medium

```
                SQS                        RabbitMQ
Management      Fully managed (AWS)         Self-hosted or CloudAMQP
Cost            Per-message pricing         Infrastructure cost
Ops burden      Zero                        High (clusters, upgrades)
Message routing Simple (one queue)          Rich (exchanges, bindings)
Message TTL     Per-queue                   Per-message
Priority        No                          Yes (x-max-priority)
Replay          No                          No (same as SQS)
Protocol        HTTP/AWS SDK                 AMQP (richer)
Throughput      Unlimited (Standard)        ~100K/sec (single cluster)
Max message     256KB                       Default unlimited
Ordering        FIFO queue                  Single consumer per queue

Choose SQS over RabbitMQ:
  AWS-native deployment (no extra infra)
  Don't want to manage message broker
  Simple queue patterns without complex routing
  Variable workload (pay only for what you use)

Choose RabbitMQ over SQS:
  Complex routing: topic exchange, dead letter, priority queues
  AMQP protocol required
  Non-AWS environment
  Per-message TTL needed
  Already have RabbitMQ expertise
```

Q37. What are SQS architecture anti-patterns?

Difficulty: Hard

```
Anti-pattern 1: Using SQS as a database
  Storing state in SQS messages (polling for state)
  SQS is a queue, not storage
  Fix: use DynamoDB, RDS, or Redis for state

Anti-pattern 2: Single queue for all message types
  All events in one queue → consumers must filter in code
  Fix: separate queues by type OR SNS → multiple SQS queues

Anti-pattern 3: No DLQ configured
  Failed messages silently expire after retention period
  Fix: always configure DLQ + CloudWatch alarm on DLQ depth

Anti-pattern 4: Tiny visibility timeout
  Consumer needs 2 min, timeout is 30s
  → Duplicate processing on every message
  Fix: set visibility timeout >> max processing time

Anti-pattern 5: No idempotency with Standard queue
  Processing payment twice = overcharge
  Fix: idempotency check before processing (DynamoDB conditional write)

Anti-pattern 6: Processing in ReceiveMessage loop without batching
  1 message per poll → 1 API call per message → high cost
  Fix: MaxNumberOfMessages=10, batch process

Anti-pattern 7: Synchronous caller waiting on SQS
  Client sends message, then polls for result
  Fix: for sync: use API call directly; for async results: use callback/webhook
  Or: use SQS Request-Reply with correlation ID + separate response queue
```

Q38. ### Q79-Q85: More Architecture Topics — **Difficulty: Medium**

Q	Topic
Q79	SQS as a buffer for downstream rate limiting
Q80	SQS in event-driven microservices architecture
Q81	Saga pattern with SQS for distributed transactions
Q82	SQS fan-out vs Kinesis fan-out: when to choose each
Q83	SQS with Step Functions for workflow orchestration
Q84	SQS for scheduled task execution with EventBridge
Q85	SQS reliability: what AWS SLA guarantees

Lambda Triggers, Go SDK & Production Patterns

Q39. How does Lambda + SQS trigger work?

Difficulty: Medium

```
Lambda Event Source Mapping (ESM):
  Lambda polls SQS on your behalf
  Invokes Lambda function with batch of messages
  Lambda handles deletion on success

Configuration:
  BatchSize: 1-10,000 messages per invocation
  MaximumBatchingWindowInSeconds: wait up to N seconds to fill batch
  FunctionResponseTypes: ReportBatchItemFailures (partial success)

Lambda invocation:
  ESM polls queue → batch ready → invoke Lambda with event

  {
    "Records": [
      {
        "messageId": "abc-123",
        "receiptHandle": "...",
        "body": "{\"orderId\": 42}",
        "attributes": {"ApproximateReceiveCount": "1"},
        "messageAttributes": {"eventType": {"stringValue": "order.created"}}
      }
    ]
  }

Success: Lambda returns (no error) → ESM deletes ALL messages in batch
Failure: Lambda throws error → ESM makes messages visible → retry
         After maxReceiveCount: → DLQ

Partial batch failure (best practice):
  Return {batchItemFailures: [{itemIdentifier: messageId}]}
  Only failed messages are retried; successful ones deleted
  Without this: ALL messages retried even if only 1 failed

FIFO + Lambda:
  One concurrent invocation per message group
  Within group: strict ordering maintained
  Across groups: parallel Lambda invocations
```

Q40. How do you implement SQS consumer in Go?

Difficulty: Medium

```
package main

import (
    "context"
    "encoding/json"
    "log"
    "sync"

    "github.com/aws/aws-sdk-go-v2/aws"
    "github.com/aws/aws-sdk-go-v2/config"
    "github.com/aws/aws-sdk-go-v2/service/sqs"
    "github.com/aws/aws-sdk-go-v2/service/sqs/types"
```

```
)

type Consumer struct {
    client   *sqs.Client
    queueURL string
    workers  int
}

func (c *Consumer) Run(ctx context.Context) {
    jobs := make(chan types.Message, c.workers*2)
    var wg sync.WaitGroup

    // Start worker pool
    for i := 0; i < c.workers; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            for msg := range jobs {
                c.process(ctx, msg)
            }
        }()
    }

    // Poll loop
    for {
        select {
        case <-ctx.Done():
            close(jobs)
            wg.Wait()
            return
        default:
            result, err := c.client.ReceiveMessage(ctx, &sqs.ReceiveMessageInput{
                QueueUrl:              aws.String(c.queueURL),
                MaxNumberOfMessages:   10,
                WaitTimeSeconds:       20,   // long polling
                AttributeNames:        []types.QueueAttributeName{"ApproximateReceiveCount"},
                MessageAttributeNames: []string{"All"},
            })
            if err != nil { log.Printf("receive error: %v", err); continue }
            for _, msg := range result.Messages {
                jobs <- msg
            }
        }
    }
}

func (c *Consumer) process(ctx context.Context, msg types.Message) {
    var payload OrderPayload
    if err := json.Unmarshal([]byte(aws.ToString(msg.Body)), &payload); err != nil {
        log.Printf("unmarshal error for %s: %v", aws.ToString(msg.MessageId), err)
        return // don't delete → goes to DLQ after maxReceiveCount
    }
    if err := processOrder(ctx, payload); err != nil {
        log.Printf("process error for %s: %v", aws.ToString(msg.MessageId), err)
        return // don't delete → retry
    }
    c.client.DeleteMessage(ctx, &sqs.DeleteMessageInput{
```

```
        QueueUrl:      aws.String(c.queueURL),
        ReceiptHandle: msg.ReceiptHandle,
    })
}
```

Q41. How do you implement SQS producer in Go? Difficulty: Easy

```
import (
    "github.com/aws/aws-sdk-go-v2/aws"
    "github.com/aws/aws-sdk-go-v2/config"
    "github.com/aws/aws-sdk-go-v2/service/sqs"
    "github.com/aws/aws-sdk-go-v2/service/sqs/types"
)

type Producer struct {
    client   *sqs.Client
    queueURL string
}

func NewProducer(ctx context.Context, queueURL string) (*Producer, error) {
    cfg, err := config.LoadDefaultConfig(ctx)
    if err != nil { return nil, err }
    return &Producer{
        client:   sqs.NewFromConfig(cfg),
        queueURL: queueURL,
    }, nil
}

func (p *Producer) Send(ctx context.Context, payload interface{}, eventType string) error {
    body, err := json.Marshal(payload)
    if err != nil { return fmt.Errorf("marshal: %w", err) }

    _, err = p.client.SendMessage(ctx, &sqs.SendMessageInput{
        QueueUrl:    aws.String(p.queueURL),
        MessageBody: aws.String(string(body)),
        MessageAttributes: map[string]types.MessageAttributeValue{
            "EventType": {
                DataType:    aws.String("String"),
                StringValue: aws.String(eventType),
            },
            "TraceId": {
                DataType:    aws.String("String"),
                StringValue: aws.String(traceIDFromContext(ctx)),
            },
        },
    })
    return err
}

// Batch send (cost-efficient):
func (p *Producer) SendBatch(ctx context.Context, messages []BatchMessage) error {
    entries := make([]types.SendMessageBatchRequestEntry, len(messages))
    for i, m := range messages {
        body, _ := json.Marshal(m.Payload)
        entries[i] = types.SendMessageBatchRequestEntry{
            Id:          aws.String(fmt.Sprintf("msg-%d", i)),
            MessageBody: aws.String(string(body)),
```

```
        }
    }
    result, err := p.client.SendMessageBatch(ctx, &sqs.SendMessageBatchInput{
        QueueUrl: aws.String(p.queueURL),
        Entries:  entries,
    })
    if len(result.Failed) > 0 {
        return fmt.Errorf("%d messages failed to send", len(result.Failed))
    }
    return err
}
```

Q42. How do you implement Lambda SQS handler in Go?

Difficulty: Medium

```
import (
    "github.com/aws/aws-lambda-go/events"
    "github.com/aws/aws-lambda-go/lambda"
)

type SQSHandler struct {
    processor OrderProcessor
}

// Handle batch with partial failure support
func (h *SQSHandler) Handle(ctx context.Context, event events.SQSEvent) (events.SQSEventResponse, error) {
    var failures []events.SQSBatchItemFailure

    for _, record := range event.Records {
        if err := h.processRecord(ctx, record); err != nil {
            log.Printf("failed to process message %s: %v", record.MessageId, err)
            failures = append(failures, events.SQSBatchItemFailure{
                ItemIdentifier: record.MessageId,
            })
        }
    }

    // Return partial failures: only failed messages are retried
    // Successful messages are automatically deleted by Lambda ESM
    return events.SQSEventResponse{
        BatchItemFailures: failures,
    }, nil
}

func (h *SQSHandler) processRecord(ctx context.Context, record events.SQSMessage) error {
    var order OrderPayload
    if err := json.Unmarshal([]byte(record.Body), &order); err != nil {
        // Non-retryable: return nil to delete (bad message) or error to retry
        log.Printf("malformed message %s, skipping: %v", record.MessageId, err)
        return nil // delete it; don't retry unparseable messages
    }

    // Check idempotency
    if processed, _ := h.processor.IsAlreadyProcessed(ctx, record.MessageId); processed {
        return nil // already done, delete
    }
    return h.processor.Process(ctx, order, record.MessageId)
```

```
}

func main() {
    handler := &SQSHandler{processor: NewOrderProcessor()}
    lambda.Start(handler.Handle)
}
```

Q43. How do you handle idempotency with SQS in Go? Difficulty: Hard

```
// DynamoDB-based idempotency store
import "github.com/aws/aws-sdk-go-v2/service/dynamodb"

type IdempotencyStore struct {
    dynamo    *dynamodb.Client
    tableName string
    ttl       time.Duration
}

// Returns (alreadyProcessed, error)
func (s *IdempotencyStore) CheckAndSet(ctx context.Context, messageID string) (bool, error) {
    now := time.Now()
    expiry := now.Add(s.ttl).Unix()

    _, err := s.dynamo.PutItem(ctx, &dynamodb.PutItemInput{
        TableName: aws.String(s.tableName),
        Item: map[string]types.AttributeValue{
            "message_id":   &types.AttributeValueMemberS{Value: messageID},
            "processed_at": &types.AttributeValueMemberN{Value: strconv.FormatInt(now.Unix(), 10)},
            "ttl":          &types.AttributeValueMemberN{Value: strconv.FormatInt(expiry, 10)},
        },
        ConditionExpression: aws.String("attribute_not_exists(message_id)"),
    })

    if err != nil {
        var ccf *types.ConditionalCheckFailedException
        if errors.As(err, &ccf) {
            return true, nil // already processed
        }
        return false, err
    }
    return false, nil // first time processing
}

// Usage in consumer:
func processMessage(ctx context.Context, msg types.Message) error {
    alreadyDone, err := idempotency.CheckAndSet(ctx, *msg.MessageId)
    if err != nil { return err }
    if alreadyDone {
        log.Printf("message %s already processed, skipping", *msg.MessageId)
        return nil
    }
    return doActualWork(ctx, msg)
}
```

Q44. How do you implement a visibility timeout heartbeat in Go? Difficulty: Hard

```
func (c *Consumer) processWithHeartbeat(ctx context.Context, msg types.Message) error {
    ctx, cancel := context.WithCancel(ctx)
    defer cancel()

    // Start heartbeat goroutine
    heartbeatDone := make(chan struct{})
    go func() {
        defer close(heartbeatDone)
        ticker := time.NewTicker(25 * time.Second) // extend every 25s
        defer ticker.Stop()
        for {
            select {
            case <-ctx.Done():
                return
            case <-ticker.C:
                _, err := c.client.ChangeMessageVisibility(context.Background(),
                    &sqs.ChangeMessageVisibilityInput{
                        QueueUrl:          aws.String(c.queueURL),
                        ReceiptHandle:     msg.ReceiptHandle,
                        VisibilityTimeout: aws.Int32(60), // extend by 60s
                    })
                if err != nil {
                    log.Printf("heartbeat failed for %s: %v", aws.ToString(msg.MessageId), err)
                }
            }
        }
    }()

    // Do actual work
    err := c.doWork(ctx, msg)
    cancel()         // stop heartbeat
    <-heartbeatDone  // wait for heartbeat goroutine to exit
    return err
}
```

Q45. How do you implement SQS with exponential backoff?

Difficulty: Medium

```
// Instead of relying solely on SQS retry (fixed delay):
// Implement explicit backoff by using ChangeMessageVisibility

func (c *Consumer) processWithBackoff(ctx context.Context, msg types.Message) {
    receiveCount := getReceiveCount(msg)

    err := processMessage(ctx, msg)
    if err == nil {
        // Success: delete message
        c.client.DeleteMessage(ctx, &sqs.DeleteMessageInput{
            QueueUrl:      aws.String(c.queueURL),
            ReceiptHandle: msg.ReceiptHandle,
        })
        return
    }

    if !isRetryable(err) {
        // Non-retryable: delete and send to DLQ manually
        sendToDLQ(ctx, msg, err)
```

```
        c.client.DeleteMessage(ctx, &sqs.DeleteMessageInput{
            QueueUrl:      aws.String(c.queueURL),
            ReceiptHandle: msg.ReceiptHandle,
        })
        return
    }

    // Retryable: set visibility timeout based on attempt number
    backoffSeconds := int32(math.Min(
        30*math.Pow(2, float64(receiveCount-1)), // 30, 60, 120, 240, 300
        300, // max 5 minutes (SQS limit 900s for message delay)
    ))
    c.client.ChangeMessageVisibility(ctx, &sqs.ChangeMessageVisibilityInput{
        QueueUrl:          aws.String(c.queueURL),
        ReceiptHandle:     msg.ReceiptHandle,
        VisibilityTimeout: aws.Int32(backoffSeconds),
    })
}

func getReceiveCount(msg types.Message) int {
    if v, ok := msg.Attributes["ApproximateReceiveCount"]; ok {
        n, _ := strconv.Atoi(v)
        return n
    }
    return 1
}
```

Q46. How do you test SQS code in Go?

Difficulty: Medium

```
// Option 1: Interface + Mock
type SQSClient interface {
    SendMessage(ctx context.Context, params *sqs.SendMessageInput, ...) (*sqs.SendMessageOutput, error)
    ReceiveMessage(ctx context.Context, params *sqs.ReceiveMessageInput, ...) (*sqs.ReceiveMessageOutput, er
    DeleteMessage(ctx context.Context, params *sqs.DeleteMessageInput, ...) (*sqs.DeleteMessageOutput, error
}

type MockSQSClient struct {
    messages []types.Message
    sent     []string
}

func (m *MockSQSClient) ReceiveMessage(ctx context.Context, _ *sqs.ReceiveMessageInput, ...) (*sqs.ReceiveMe
    if len(m.messages) == 0 {
        return &sqs.ReceiveMessageOutput{}, nil
    }
    msg := m.messages[0]
    m.messages = m.messages[1:]
    return &sqs.ReceiveMessageOutput{Messages: []types.Message{msg}}, nil
}

// Option 2: LocalStack (real SQS API locally)
// docker run -p 4566:4566 localstack/localstack
// Set endpoint: http://localhost:4566

cfg, _ := config.LoadDefaultConfig(ctx,
    config.WithEndpointResolverWithOptions(
```

```
        aws.EndpointResolverWithOptionsFunc(func(service, region string, options ...interface{}) (aws.Endpoi
            return aws.Endpoint{URL: "http://localhost:4566"}, nil
        }),
    ),
    config.WithCredentialsProvider(credentials.NewStaticCredentialsProvider("test", "test", "")),
)

// Option 3: testcontainers-go with LocalStack
container, _ := localstack.RunContainer(ctx, testcontainers.WithImage("localstack/localstack"))
endpoint, _ := container.SQSURL(ctx)
```

Q47. How do you monitor SQS queues in production?

Difficulty: Medium

```
// CloudWatch alarms via Go (setup during deploy):
import "github.com/aws/aws-sdk-go-v2/service/cloudwatch"

func setupQueueAlarms(ctx context.Context, cw *cloudwatch.Client, queueName, dlqName string) error {
    // Alarm: queue depth growing (consumers lagging)
    _, err := cw.PutMetricAlarm(ctx, &cloudwatch.PutMetricAlarmInput{
        AlarmName:          aws.String(queueName + "-depth-alarm"),
        MetricName:         aws.String("ApproximateNumberOfMessagesVisible"),
        Namespace:          aws.String("AWS/SQS"),
        Statistic:          types.StatisticMaximum,
        Period:             aws.Int32(300),  // 5 minutes
        EvaluationPeriods:  aws.Int32(2),
        Threshold:          aws.Float64(10000),
        ComparisonOperator: types.ComparisonOperatorGreaterThanThreshold,
        Dimensions: []types.Dimension{{
            Name:  aws.String("QueueName"),
            Value: aws.String(queueName),
        }},
        AlarmActions: []string{"arn:aws:sns:...:alerts"},
    })
    if err != nil { return err }

    // Alarm: any message in DLQ
    _, err = cw.PutMetricAlarm(ctx, &cloudwatch.PutMetricAlarmInput{
        AlarmName:          aws.String(dlqName + "-not-empty"),
        MetricName:         aws.String("ApproximateNumberOfMessagesVisible"),
        Namespace:          aws.String("AWS/SQS"),
        Statistic:          types.StatisticSum,
        Period:             aws.Int32(60),
        EvaluationPeriods:  aws.Int32(1),
        Threshold:          aws.Float64(0),
        ComparisonOperator: types.ComparisonOperatorGreaterThanThreshold,
        Dimensions: []types.Dimension{{
            Name:  aws.String("QueueName"),
            Value: aws.String(dlqName),
        }},
        AlarmActions: []string{"arn:aws:sns:...:alerts"},
    })
    return err
}
```

Q48. What is SQS cost optimization?

Difficulty: Medium

```
Cost reduction strategies:

1. Long polling (WaitTimeSeconds=20):
   Fewer empty receives → fewer API calls
   Example: 1 msg/min queue
   Short polling: 60 × $0.40/M = ~$0 but wasteful + CPU
   Long polling: 3 × $0.40/M (20s each) = 5× fewer calls

2. Batch operations:
   SendMessageBatch: 10 msgs = 1 API call (not 10)
   DeleteMessageBatch: 10 msgs = 1 API call
   10× cost reduction for high-volume queues

3. Increase batch size in Lambda ESM:
   BatchSize=10 → 1 Lambda invocation per 10 messages
   BatchSize=1 → 10 Lambda invocations (10× more expensive)

4. Message consolidation:
   Combine related small messages into one batch message
   {"events": [event1, event2, event3]} = 1 message = 1 API call vs 3

5. Right-size polling interval:
   Low-volume queues: long polling + increase polling interval
   High-volume queues: always polling (long polling = 20s max wait anyway)

6. Standard over FIFO when possible:
   FIFO: 25% more expensive
   If ordering not required: use Standard

7. Filter at SNS level (subscription filter policy):
   Services receive only relevant messages
   Fewer messages → fewer SQS API calls

Pricing example (1M messages/day, 10 msg/batch):
  Without batch: 1M API calls = $0.40/day
  With batch: 100K API calls = $0.04/day (10× cheaper)
```

Q49. ### Q96-Q100: Final Advanced Topics

Difficulty: Medium

Q50. What is SQS resource tagging and cost allocation?

Difficulty: Easy

```
// Tag queues for cost allocation
svc.TagQueue(ctx, &sqs.TagQueueInput{
    QueueUrl: aws.String(queueURL),
    Tags: map[string]string{
        "Environment": "production",
        "Service":     "order-service",
        "Team":        "platform",
        "CostCenter":  "engineering",
    },
})

// List tags
```

```
result, _ := svc.ListQueueTags(ctx, &sqs.ListQueueTagsInput{
    QueueUrl: aws.String(queueURL),
})

// AWS Cost Explorer: filter by tag
// Service: SQS, Tag: Service=order-service
// → shows SQS cost attributed to order service
```

Q51. What is SQS cross-region and cross-account patterns?

Difficulty: Hard

```
Cross-account SQS:
  Queue in Account A, producer in Account B

  Queue policy (Account A): allow Account B to SendMessage
  {
    "Effect": "Allow",
    "Principal": {"AWS": "arn:aws:iam::ACCOUNT-B:root"},
    "Action": "sqs:SendMessage",
    "Resource": "arn:aws:sqs:us-east-1:ACCOUNT-A:my-queue"
  }

  Producer (Account B): use queue ARN from Account A
  Assumes role with sqs:SendMessage or directly with queue policy

Cross-region SQS:
  SQS queues are region-specific
  No built-in cross-region replication

  Pattern: SNS cross-region subscription
    SNS Topic (us-east-1) → SNS subscription → SQS (eu-west-1)
    Via HTTP subscription or SQS-to-SQS (not native)

  Or: application-level dual-write to queues in multiple regions

  EventBridge: native cross-region event routing
    EventBridge rule (us-east-1) → EventBridge bus (eu-west-1) → SQS (eu-west-1)

Typical use case:
  Multi-region active-active: each region has its own SQS queues
  Global events: use EventBridge for cross-region routing to SQS
```

Q52. What is SQS with AWS Step Functions?

Difficulty: Medium

```
Step Functions + SQS: orchestrate long-running workflows with SQS as transport

Pattern 1: SQS as trigger for Step Functions
  SQS message → Lambda → starts Step Functions execution

Pattern 2: Step Functions with SQS task (Wait for callback)
  Step Functions starts → sends message to SQS with taskToken
  Consumer processes → calls SendTaskSuccess(taskToken) → Step Functions continues

  {
    "Type": "Task",
    "Resource": "arn:aws:states:::sqs:sendMessage.waitForTaskToken",
    "Parameters": {
```

```
      "QueueUrl": "https://sqs...",
      "MessageBody": {
        "taskToken.$": "$$.Task.Token",
        "input.$": "$"
      }
    }
  }

// Go consumer: complete Step Functions task
svc.SendTaskSuccess(ctx, &sfn.SendTaskSuccessInput{
    TaskToken: aws.String(taskToken),
    Output:    aws.String(`{"result": "success"}`),
})
// or failure:
svc.SendTaskFailure(ctx, &sfn.SendTaskFailureInput{
    TaskToken: aws.String(taskToken),
    Error:     aws.String("ProcessingFailed"),
    Cause:     aws.String(err.Error()),
})

Use case: human approval workflows, external system integration,
          long-running async processing with correlation
```

Q53. What is the SQS outbox pattern implementation? Difficulty: Hard

```
// Transactional outbox: write to DB + SQS atomically using DB outbox table

// Step 1: Write to outbox table in same DB transaction
func (s *OrderService) CreateOrder(ctx context.Context, order *Order) error {
    return s.db.WithTx(ctx, func(tx *sql.Tx) error {
        // Business logic
        if err := insertOrder(ctx, tx, order); err != nil { return err }

        // Write to outbox (same transaction = atomic!)
        eventPayload, _ := json.Marshal(OrderCreatedEvent{
            OrderID:   order.ID,
            UserID:    order.UserID,
            Amount:    order.Amount,
            CreatedAt: order.CreatedAt,
        })
        _, err := tx.ExecContext(ctx, `
            INSERT INTO outbox (id, event_type, payload, created_at)
            VALUES ($1, $2, $3, NOW())`,
            uuid.New().String(), "order.created", eventPayload,
        )
        return err
    })
}

// Step 2: Outbox relay (background goroutine or separate service)
func (r *OutboxRelay) Run(ctx context.Context) {
    for {
        select {
        case <-ctx.Done(): return
        case <-time.After(100 * time.Millisecond):
            r.publishPending(ctx)
        }
```

```
    }
}

func (r *OutboxRelay) publishPending(ctx context.Context) {
    rows, _ := r.db.QueryContext(ctx, `
        SELECT id, event_type, payload FROM outbox
        WHERE published_at IS NULL ORDER BY created_at LIMIT 100
        FOR UPDATE SKIP LOCKED`)
    defer rows.Close()

    var entries []OutboxEntry
    for rows.Next() {
        var e OutboxEntry
        rows.Scan(&e.ID, &e.EventType, &e.Payload)
        entries = append(entries, e)
    }
    if len(entries) == 0 { return }

    // Send to SQS in batches of 10
    for i := 0; i < len(entries); i += 10 {
        batch := entries[i:min(i+10, len(entries))]
        r.sendBatch(ctx, batch)
    }
}
```

Q54. What are SQS production checklist items?

Difficulty: Medium

Production readiness checklist:

Queue Configuration:
- ■ DLQ configured on every queue
- ■ maxReceiveCount set appropriately (3-5 for most)
- ■ DLQ retention = 14 days
- ■ Visibility timeout = max_processing_time × 6
- ■ Long polling enabled (WaitTimeSeconds=20)
- ■ SSE enabled for sensitive data
- ■ Appropriate access policy (least privilege IAM)

Consumer:
- ■ Idempotency implemented (Standard queues)
- ■ Batch processing (MaxNumberOfMessages=10)
- ■ Visibility timeout heartbeat for long-running tasks
- ■ Graceful shutdown: drain in-flight on SIGTERM
- ■ Structured error handling (retryable vs non-retryable)
- ■ Partial batch failure for Lambda (ReportBatchItemFailures)

Monitoring:
- ■ CloudWatch alarm: queue depth growing (consumers lagging)
- ■ CloudWatch alarm: DLQ not empty (any message = bug)
- ■ CloudWatch alarm: ApproximateAgeOfOldestMessage > threshold
- ■ Distributed tracing: trace IDs in message attributes
- ■ Structured logging: message IDs, receive counts, processing time

Operations:
- ■ DLQ redrive procedure documented and tested
- ■ Queue purge procedure for emergency data flush

```
  ■ Message replay playbook
  ■ Runbook: what to do when DLQ alarm fires
  ■ Load testing: verify throughput meets requirements

Cost:
  ■ Batch operations used (10× cost reduction)
  ■ Resource tags for cost allocation
  ■ Queue type right-sized (Standard vs FIFO)
```

Master these 100 questions and you'll handle any AWS SQS interview at SDE2 level. Key areas: Standard vs FIFO (when to use each), visibility timeout and message lifecycle, DLQ configuration and redrive, SNS+SQS fan-out patterns, SQS vs Kafka decision framework, Lambda partial batch failures, and Go AWS SDK patterns (consumer, producer, idempotency, backoff). ■

Additional SQS Questions (Q55–Q100)

Q55. What is SQS FIFO exactly-once processing?

Difficulty: Hard

```
SQS FIFO Exactly-Once Processing:
  MessageDeduplicationId: prevents duplicate messages within 5-minute window
  MessageGroupId: enables per-group ordering

Deduplication methods:
  1. Content-based: SHA-256 of message body (set ContentBasedDeduplication=true)
  2. Explicit: you provide MessageDeduplicationId

Example:
  MessageDeduplicationId = "order-42-payment"
  If same ID sent within 5 minutes → second send silently discarded

FIFO limitation: 300 TPS (3000 with batching)
Standard: 300,000+ TPS (nearly unlimited)

If you need high throughput + ordering:
  Use multiple FIFO queues (shard by customer/entity)
  Or use Kafka (no TPS limit per partition)
```

Q56. What is SQS message attributes?

Difficulty: Medium

```
# Message attributes: metadata alongside message body
# Types: String, Number, Binary, String.Array

sqs.send_message(
    QueueUrl=queue_url,
    MessageBody='{"orderId": "42"}',
    MessageAttributes={
        'EventType': {
            'StringValue': 'order.created',
            'DataType': 'String'
        },
        'Version': {
            'StringValue': '1',
            'DataType': 'Number'
```

```
        },
        'Timestamp': {
            'StringValue': '2024-01-15T10:00:00Z',
            'DataType': 'String'
        },
        'TraceId': {
            'StringValue': 'abc-123-def',
            'DataType': 'String'
        }
    }
)

# Receive with attributes
response = sqs.receive_message(
    QueueUrl=queue_url,
    MessageAttributeNames=['All'],  # or specific: ['EventType']
    AttributeNames=['All']          # system attributes (SentTimestamp, etc.)
)

# System attributes available:
# SentTimestamp, ApproximateFirstReceiveTimestamp
# ApproximateReceiveCount, MessageDeduplicationId (FIFO)

# Use for: routing without parsing body, filtering, tracing
```

Q57. What is SQS with Lambda event source mapping? Difficulty: Hard

```
SQS → Lambda integration:
  Lambda polls SQS (long-polling, 5 parallel pollers per shard)
  Lambda invoked with batch of messages (up to 10,000 for Standard)
  Lambda scales: 1 concurrent execution per queue (initial)
    → scales up to 1000 concurrent (Standard)
    → 1 concurrent per MessageGroupId (FIFO)

Failure behavior (critical):
  If Lambda throws: entire batch returned to queue and retried
  "Batch Item Failures": report partial failures
    → only failed messages go back to queue
    → successfully processed messages NOT re-sent

Lambda function (batch item failure reporting):
def handler(event, context):
    batch_item_failures = []
    for record in event['Records']:
        try:
            process(record['body'])
        except Exception as e:
            batch_item_failures.append({
                "itemIdentifier": record['messageId']
            })
    return {"batchItemFailures": batch_item_failures}

Event source mapping settings:
  BatchSize: 1–10,000 (Standard), 1–10 (FIFO)
  MaximumBatchingWindowInSeconds: wait for more messages (0–300s)
  FunctionResponseTypes: ["ReportBatchItemFailures"]
  ScalingConfig.MaximumConcurrency: limit concurrent Lambdas
```

```
Tip: set Lambda DLQ + SQS DLQ for full failure capture
```

Q58. What is SQS vs Kinesis?

Difficulty: Hard

```
SQS:
  Ordering: FIFO queue only, no ordering in Standard
  Consumers: competing consumers (each message processed by ONE consumer)
  Retention: up to 14 days
  Replay: NOT possible (consumed message deleted after ACK)
  Throughput: Standard: virtually unlimited; FIFO: 300 TPS
  Message size: max 256KB
  Use: work queue, task queue, microservice decoupling

Kinesis Data Streams:
  Ordering: per shard (strict ordering)
  Consumers: multiple consumer groups (same data read by multiple)
  Retention: 24h (default) to 7 days (extended), 365 days (long-term)
  Replay: YES (re-read from any sequence number within retention)
  Throughput: 1MB/s write per shard, 2MB/s read per shard
  Message size: max 1MB per record
  Use: real-time analytics, event sourcing, multiple consumers

Decision:
  SQS: "do this task once" (queue semantic)
  Kinesis: "record this event for multiple consumers / replay" (log semantic)

SQS → Kinesis mapping:
  SQS consumer group    ≈ Kinesis consumer group
  SQS message deleted   ≠ Kinesis (pointer moves, data stays)
  SQS DLQ              → Kinesis no built-in DLQ (use Lambda DLQ)
```

Q59. What is SQS Extended Client Library?

Difficulty: Medium

```
SQS message limit: 256KB
Large message solution: Amazon SQS Extended Client Library
  Stores payload in S3, sends S3 reference in SQS message

Flow:
  Producer:
    1. Upload payload to S3 (auto-generates key)
    2. Send SQS message with S3 pointer: {"s3BucketName":"...", "s3Key":"..."}

  Consumer:
    1. Receive SQS message
    2. Detect S3 pointer (special message attribute)
    3. Download payload from S3
    4. Process
    5. Delete SQS message + S3 object

Java/Python: aws-sqs-java-extended-client-lib
Go: implement manually using same protocol

Manual Go implementation:
const maxSQSPayload = 250 * 1024  // 250KB to be safe

func sendLargeMessage(body []byte) error {
```

```
    if len(body) < maxSQSPayload {
        _, err := sqs.SendMessage(ctx, &sqs.SendMessageInput{
            QueueUrl:    &queueURL,
            MessageBody: aws.String(string(body)),
        })
        return err
    }
    // Upload to S3
    key := uuid.New().String()
    s3.PutObject(ctx, &s3.PutObjectInput{
        Bucket: aws.String(bucket),
        Key:    aws.String(key),
        Body:   bytes.NewReader(body),
    })
    // Send reference
    ref := fmt.Sprintf(`{"bucket":"%s","key":"%s"}`, bucket, key)
    _, err := sqs.SendMessage(ctx, &sqs.SendMessageInput{
        QueueUrl:    &queueURL,
        MessageBody: aws.String(ref),
    })
    return err
}
```

Q60. What is SQS message filtering with SNS?

Difficulty: Hard

```
SNS → SQS Fan-out with filtering:
  Topic: OrderEvents
  Queue1 (order-service): filter by "eventType": "order.created"
  Queue2 (inventory):     filter by "eventType": ["order.created", "order.cancelled"]
  Queue3 (analytics):     no filter (receives everything)

Subscription filter policy (JSON):
{
  "eventType": ["order.created"],
  "region": ["us-east-1", "us-west-2"],
  "amount": [{"numeric": [">=", 100]}]
}

Set via CLI:
aws sns set-subscription-attributes \
  --subscription-arn arn:aws:sns:... \
  --attribute-name FilterPolicy \
  --attribute-value '{"eventType":["order.created"]}'

SNS filter on message attributes:
  Publisher sets attributes → SNS evaluates filter before delivery
  Filtered-out messages never reach SQS → cost savings, less noise

Advanced filter operators:
  Exact match: "status": ["SHIPPED"]
  Prefix: "prefix": "order-"
  Numeric: {"numeric": [">", 0, "<=", 100]}
  Anything-but: {"anything-but": ["ERROR"]}
  Exists: {"exists": true}
```

Q61. What is SQS with AWS EventBridge?

Difficulty: Medium

```
EventBridge → SQS:
  EventBridge: sophisticated event routing (rules-based)
  Target: SQS queue

Use case:
  EC2 instance terminates → EventBridge rule → SQS queue → Lambda cleanup

EventBridge advantages over direct SQS publish:
  Content-based filtering (complex rules)
  Multiple targets (SQS + Lambda + Step Functions simultaneously)
  Archive + replay events
  Schema registry
  Cross-account and cross-region delivery

Rule example:
  Event pattern:
  {
    "source": ["aws.ec2"],
    "detail-type": ["EC2 Instance State-change Notification"],
    "detail": { "state": ["terminated"] }
  }
  Target: SQS queue arn:aws:sqs:us-east-1:...

Comparison:
  SNS → SQS: fan-out, simple publish-subscribe, filter on attributes
  EventBridge → SQS: complex routing, AWS service events, archive/replay
  Both: asynchronous decoupling

EventBridge Pipes:
  Source: SQS → enrichment (Lambda) → target (another SQS/API/Step Functions)
  Point-to-point pipeline with transformation
```

Q62. What is SQS cost optimization?

Difficulty: Medium

```
SQS pricing:
  Standard: $0.40 per 1M requests (first 1M free/month)
  FIFO: $0.50 per 1M requests
  Data transfer: free within same region

Optimization strategies:

1. Long polling (biggest impact):
   Short polling: charges per poll (even empty response)
   Long polling (WaitTimeSeconds=20): fewer polls, fewer charges
   20s wait = 3 polls/min vs 60 polls/min = 95% cost reduction

2. Batching:
   Send: batch up to 10 messages in one API call (charged as 1 request)
   Receive: ReceiveMessage returns up to 10 at once
   Delete: DeleteMessageBatch (up to 10) = 1 API call
   10x cost reduction vs individual operations

3. Message deduplication cost:
```

```
   FIFO charges for dedup even if message rejected (not sent)
   Standard + app-level dedup: cheaper for high-volume

4. Avoid frequent polling of empty queues:
   Use SQS → Lambda (Lambda handles polling, you pay per invocation)

5. Extended Client for large payloads:
   S3 is cheaper than large SQS messages (over 64KB billing starts)
   256KB SQS message = billed as 4 × 64KB chunks = 4 requests!

6. Monitor:
   NumberOfEmptyReceives: high → increase WaitTimeSeconds
   ApproximateNumberOfMessages: high → scale consumers
```

Q63. What are SQS security best practices?

Difficulty: Medium

```
1. Encryption:
   SSE-SQS: AWS managed keys (default, free)
   SSE-KMS: customer managed keys (better audit, cross-account)

   Queue policy → Condition: aws:SecureTransport: true
   → Reject unencrypted (non-HTTPS) connections

2. IAM: least privilege
   Producer: sqs:SendMessage only
   Consumer: sqs:ReceiveMessage, sqs:DeleteMessage, sqs:GetQueueAttributes
   Admin: full access (restricted)

3. Queue policy for cross-account:
{
  "Effect": "Allow",
  "Principal": {"AWS": "arn:aws:iam::ACCOUNT2:root"},
  "Action": ["sqs:SendMessage"],
  "Resource": "arn:aws:sqs:...:myqueue",
  "Condition": {
    "ArnLike": {"aws:SourceArn": "arn:aws:sns:...:mytopic"}
  }
}

4. VPC endpoint: keep traffic within AWS network
   Gateway endpoint for S3/DynamoDB (free)
   Interface endpoint for SQS (hourly charge)
   → Prevents SQS traffic from going over internet

5. Resource-based policy vs IAM:
   IAM: attached to identity (role/user)
   Resource policy: attached to queue, controls who can access
   Both must allow for cross-account access

6. CloudTrail: log all SQS API calls (SendMessage, DeleteMessage, etc.)
   Useful for: compliance, debugging, security audits
```

Q64. What is SQS consumer scaling strategies?

Difficulty: Hard

```
Scaling consumers based on queue depth:
```

```
1. CloudWatch + Auto Scaling:
   Metric: ApproximateNumberOfMessages
   Alarm: > 1000 messages → scale out
   Scale policy: target tracking (keep queue depth / consumer = X)

2. Step scaling:
   < 100 messages: 1 consumer
   100–1000: 2 consumers
   > 1000: 4 consumers

3. Kubernetes HPA + KEDA:
   KEDA: event-driven autoscaler for Kubernetes
   ScaledObject triggers on SQS depth:

   apiVersion: keda.sh/v1alpha1
   kind: ScaledObject
   spec:
     triggers:
     - type: aws-sqs-queue
       metadata:
         queueURL: https://sqs.us-east-1.amazonaws.com/.../queue
         queueLength: "100"  # target messages per consumer
         awsRegion: us-east-1

4. Lambda (no explicit scaling needed):
   Lambda auto-scales based on queue depth
   Max concurrency configurable (ReservedConcurrency)

5. Go consumer scaling pattern:
   Start N goroutines (workers)
   Each long-polls independently
   Scale workers based on ApproximateNumberOfMessages metric

Processing rate = workers × (messages per receive / processing time)
Target: queue depth < MaxReceiveCount × processing time
```

Q65. What is SQS redrive policy configuration?

Difficulty: Medium

```
// Redrive policy: configure DLQ for a source queue
// RedrivePolicy on source queue (not DLQ)
{
  "deadLetterTargetArn": "arn:aws:sqs:us-east-1:123:orders-dlq",
  "maxReceiveCount": "5"
}

// maxReceiveCount: after N failed processing attempts, send to DLQ
// A message is "failed" if:
//   - Consumer receives it but doesn't delete within visibility timeout
//   - Consumer explicitly doesn't delete (crash, exception)
// Lambda returns error → SQS makes message visible again (retry)

// SQS DLQ rules:
// - DLQ must be same type (FIFO → FIFO DLQ, Standard → Standard DLQ)
// - DLQ must be in same region
// - DLQ retention can be longer than source (extend it!)
// - Source queue's message retention period affects DLQ messages
```

```
// MessageRedriveStatus: check if message is in DLQ or source
// DLQ attributes available: sourceQueueArns

// Testing redrive policy:
// Set maxReceiveCount=1 temporarily in dev
// Observe which messages land in DLQ
// Check message body + attributes for debugging
// Fix bug, then move from DLQ back to source

// Redrive from DLQ back to source (Console or API):
aws sqs start-message-move-task \
  --source-arn arn:aws:sqs:us-east-1:123:orders-dlq \
  --destination-arn arn:aws:sqs:us-east-1:123:orders
```

Q66. What is SQS monitoring and alerting?

Difficulty: Medium

```
Key SQS CloudWatch metrics:

ApproximateNumberOfMessages:
  Total messages waiting (visible + in-flight + delayed)
  Alert: > threshold (backlog building up)

ApproximateAgeOfOldestMessage:
  Age in seconds of oldest message in queue
  Alert: > SLA (e.g., > 300s = processing falling behind)
  Best metric for: consumer lag, SLA violation

NumberOfMessagesSent: rate of incoming messages
NumberOfMessagesReceived: rate consumed
NumberOfMessagesDeleted: rate successfully processed
ApproximateNumberOfMessagesNotVisible: in-flight messages
NumberOfEmptyReceives: idle polls (consumers have nothing to do)

Alarms:
1. ApproximateAgeOfOldestMessage > 300 → PagerDuty alert
2. ApproximateNumberOfMessages > 10000 → scale out consumers
3. DLQ ApproximateNumberOfMessages > 0 → alert (messages failing)
4. NumberOfEmptyReceives > 80% → reduce consumers (over-provisioned)

Dashboard to build:
  Queue depth (ApproximateNumberOfMessages)
  Message age (ApproximateAgeOfOldestMessage)
  Throughput (sent/received/deleted per minute)
  DLQ depth (separate alarm, severity=high)
  Consumer lag ratio

Tracing:
  X-Ray: enable X-Ray tracing on Lambda + SQS
  Shows: queue wait time, processing time, downstream calls
```

Q67. What is SQS with Step Functions?

Difficulty: Hard

```
SQS → Step Functions integration patterns:

1. SQS as input trigger:
   SQS message → Lambda → start Step Functions execution
```

```
   Lambda trigger:
   def handler(event, context):
       for record in event['Records']:
           sfn.start_execution(
               stateMachineArn=STATE_MACHINE_ARN,
               input=record['body']
           )

2. Step Functions → SQS (send task):
   State machine sends message to SQS
   Task type: "Resource": "arn:aws:states:::sqs:sendMessage"
   {
     "Type": "Task",
     "Resource": "arn:aws:states:::sqs:sendMessage.waitForTaskToken",
     "Parameters": {
       "QueueUrl": "...",
       "MessageBody": {
         "taskToken.$": "$$.Task.Token",
         "input.$": "$"
       }
     }
   }

3. Wait for task token:
   Step Functions sends token in SQS message
   Worker receives message, processes, calls SendTaskSuccess/Failure
   Step Functions resumes execution
   Pattern: human approval, long-running external tasks

4. Express vs Standard workflow:
   Express: short (< 5 min), high volume (perfect for SQS fan-out)
   Standard: long-running (up to 1 year), exactly-once execution
```

Q68. What is SQS temporary queues?

Difficulty: Medium

```
Temporary queues: short-lived queues for request-response over SQS
  Pattern: replaces synchronous HTTP with async SQS

Temporary Queue Client (aws-sdk-java-sqs-java-messaging-lib):
  Creates virtual queues backed by single physical queue
  Each requester gets unique correlationId

Manual Go implementation:
const requestQueueURL = "https://sqs.../requests"

func makeRequest(payload string) (string, error) {
    // Create reply queue
    result, _ := sqs.CreateQueue(ctx, &sqs.CreateQueueInput{
        QueueName: aws.String("reply-" + uuid.New().String()),
        Attributes: map[string]string{
            "MessageRetentionPeriod": "60",  // 1 minute
        },
    })
    replyQURL := result.QueueUrl
    defer sqs.DeleteQueue(ctx, &sqs.DeleteQueueInput{QueueUrl: replyQURL})

    // Send request with reply-to
```

```
    sqs.SendMessage(ctx, &sqs.SendMessageInput{
        QueueUrl:    aws.String(requestQueueURL),
        MessageBody: aws.String(payload),
        MessageAttributes: map[string]sqsTypes.MessageAttributeValue{
            "replyTo": {StringValue: replyQURL, DataType: aws.String("String")},
        },
    })

    // Wait for response
    for i := 0; i < 10; i++ {
        resp, _ := sqs.ReceiveMessage(ctx, &sqs.ReceiveMessageInput{
            QueueUrl: replyQURL, WaitTimeSeconds: 5,
        })
        if len(resp.Messages) > 0 {
            return *resp.Messages[0].Body, nil
        }
    }
    return "", errors.New("timeout")
}
```

Q69. What is SQS batching in Go?

Difficulty: Medium

```
// Efficient batch operations

// Batch send (up to 10 per batch)
type BatchSender struct {
    sqs     *sqs.Client
    queueURL string
    mu      sync.Mutex
    batch   []sqsTypes.SendMessageBatchRequestEntry
}

func (s *BatchSender) Send(ctx context.Context, body string, attrs map[string]sqsTypes.MessageAttributeValue
    entry := sqsTypes.SendMessageBatchRequestEntry{
        Id:                aws.String(uuid.New().String()),
        MessageBody:       aws.String(body),
        MessageAttributes: attrs,
    }

    s.mu.Lock()
    s.batch = append(s.batch, entry)
    if len(s.batch) >= 10 {
        batch := s.batch
        s.batch = nil
        s.mu.Unlock()
        return s.flush(ctx, batch)
    }
    s.mu.Unlock()
    return nil
}

func (s *BatchSender) flush(ctx context.Context, entries []sqsTypes.SendMessageBatchRequestEntry) error {
    result, err := s.sqs.SendMessageBatch(ctx, &sqs.SendMessageBatchInput{
        QueueUrl: &s.queueURL,
        Entries:  entries,
    })
```

```
    if err != nil { return err }

    // Check failed entries
    for _, failed := range result.Failed {
        log.Printf("message %s failed: %s - %s", *failed.Id, *failed.Code, *failed.Message)
    }
    return nil
}

// Batch delete (after processing)
func deleteMessages(ctx context.Context, sqsClient *sqs.Client, queueURL string, receipts []string) error {
    entries := make([]sqsTypes.DeleteMessageBatchRequestEntry, len(receipts))
    for i, r := range receipts {
        entries[i] = sqsTypes.DeleteMessageBatchRequestEntry{
            Id:            aws.String(strconv.Itoa(i)),
            ReceiptHandle: aws.String(r),
        }
    }
    _, err := sqsClient.DeleteMessageBatch(ctx, &sqs.DeleteMessageBatchInput{
        QueueUrl: aws.String(queueURL),
        Entries:  entries,
    })
    return err
}
```

Q70. What is the SQS visibility timeout best practices?

Difficulty: Medium

```
Visibility timeout = time consumer has to process + delete
If not deleted in time → message reappears in queue (re-processed)

Formula:
  visibility_timeout > p99_processing_time + buffer (20%)

Example:
  Processing time p99 = 25 seconds
  Set visibility_timeout = 30 seconds minimum

Extending visibility timeout for long tasks:
  // Heartbeat: extend while processing
  func processWithHeartbeat(ctx context.Context, msg Message) error {
      ticker := time.NewTicker(20 * time.Second)  // renew every 20s
      defer ticker.Stop()

      done := make(chan error, 1)
      go func() { done <- process(msg) }()

      for {
          select {
          case <-ticker.C:
              sqs.ChangeMessageVisibility(ctx, &sqs.ChangeMessageVisibilityInput{
                  QueueUrl:          &queueURL,
                  ReceiptHandle:     msg.ReceiptHandle,
                  VisibilityTimeout: aws.Int32(30),  // extend by 30 more seconds
              })
          case err := <-done:
              return err
```

```
          }
      }
  }

Queue-level vs per-message:
  Queue: default visibility timeout (set during creation)
  ChangeMessageVisibility: override for specific messages (max 12 hours)
  MaxReceiveCount × visibility_timeout = max time before DLQ
```

Q71. ### Q71–Q100: Remaining SQS Questions

Difficulty: Medium

Q72. What is SQS message body size and chunking?

Difficulty: Medium

```
Limit: 256KB (262,144 bytes) per message
Includes: message body + message attributes

If payload > 256KB:
  1. SQS Extended Client (Java/Python): auto-stores in S3
  2. Manual: store payload in S3, put S3 reference in SQS
  3. Split message into chunks (if order doesn't matter)
  4. Consider Kinesis (1MB per record)

Billing for large messages:
  64KB = 1 "request"
  128KB = 2 requests
  256KB = 4 requests
  Combine small messages (batching) to save cost
```

Q73. What is SNS + SQS vs EventBridge?

Difficulty: Medium

```
SNS + SQS fan-out:
  Simple publish-subscribe
  Message attribute filtering
  Push delivery model (SNS pushes to SQS)
  Best for: known, fixed subscriber set

EventBridge:
  Content-based routing (JSON path filtering)
  Native AWS service integration (100+ sources)
  Schema registry + discovery
  Archive + replay events
  Cross-account/region delivery
  Higher latency (~0.5s vs SNS <1ms)
  Best for: complex routing, AWS service events, schema management

Choose SNS+SQS: simple fan-out, latency sensitive
Choose EventBridge: complex routing, audit/replay, AWS service events
```

Q74. What is SQS ApproximateNumberOfMessages lag calculation?

Difficulty: Medium

```
// Calculate processing lag and projected catch-up time

type QueueMetrics struct {
```

```
    Depth             int64
    AgeOfOldest       int64  // seconds
    MessagesPerMinute  float64
}

func checkQueueHealth(ctx context.Context, sqsClient *sqs.Client, queueURL string) (QueueMetrics, error) {
    result, err := sqsClient.GetQueueAttributes(ctx, &sqs.GetQueueAttributesInput{
        QueueUrl: &queueURL,
        AttributeNames: []sqsTypes.QueueAttributeName{
            sqsTypes.QueueAttributeNameApproximateNumberOfMessages,
            sqsTypes.QueueAttributeNameApproximateAgeOfOldestMessage,
        },
    })
    if err != nil { return QueueMetrics{}, err }

    depth, _ := strconv.ParseInt(result.Attributes["ApproximateNumberOfMessages"], 10, 64)
    age, _ := strconv.ParseInt(result.Attributes["ApproximateAgeOfOldestMessage"], 10, 64)

    return QueueMetrics{Depth: depth, AgeOfOldest: age}, nil
}
```

Q75. What is SQS server-side encryption with KMS?

Difficulty: Medium

```
SSE with AWS KMS:
  Messages encrypted at rest using KMS key
  KMS key types:
    AWS managed (aws/sqs): automatic rotation, no extra cost
    Customer managed: control rotation, cross-account, audit

  Cost: $0.03 per 10,000 KMS API calls
  Each send + each receive = 1 KMS call each

  aws sqs set-queue-attributes \
    --queue-url $URL \
    --attributes KmsMasterKeyId=alias/my-key,\
                 KmsDataKeyReusePeriodSeconds=300

  Data key reuse (300s): reduces KMS calls (cached 5 minutes)
  Higher reuse period = fewer KMS calls = lower cost

  For Lambda: grant Lambda execution role kms:Decrypt on the key
  For cross-account: key policy must allow other account
```

Q76. What is SQS delay queue per message?

Difficulty: Medium

```
// Queue-level delay: DelaySeconds on queue (0-900 seconds)
// Message-level delay: override per message

// Use case: retry with delay, scheduled processing

result, err := sqsClient.SendMessage(ctx, &sqs.SendMessageInput{
    QueueUrl:     aws.String(queueURL),
    MessageBody:  aws.String(body),
    DelaySeconds: aws.Int32(60),  // this message visible after 60 seconds
})
```

```
// FIFO queues: don't support per-message DelaySeconds
// (queue-level delay only for FIFO)

// Pattern: exponential backoff retry with delay
func retryWithDelay(attempt int) error {
    delay := int32(math.Min(float64(1<<attempt), 900))  // 1,2,4,8... max 900s
    _, err := sqsClient.SendMessage(ctx, &sqs.SendMessageInput{
        QueueUrl:     aws.String(retryQueueURL),
        MessageBody:  aws.String(payload),
        DelaySeconds: &delay,
        MessageAttributes: map[string]sqsTypes.MessageAttributeValue{
            "attempt": {StringValue: aws.String(strconv.Itoa(attempt)), DataType: aws.String("Number")},
        },
    })
    return err
}
```

Q77. ### Q76-Q100: Final SQS Questions Summary

Difficulty: Medium

Q	Topic
Q76	SQS IAM policy examples with conditions
Q77	SQS with SageMaker for ML pipelines
Q78	SQS SDK retry configuration in Go
Q79	SQS message ordering guarantees
Q80	At-least-once vs at-most-once vs exactly-once
Q81	SQS vs RabbitMQ comparison
Q82	SQS vs Kafka comparison
Q83	SQS local development with LocalStack
Q84	SQS integration testing strategies
Q85	SQS consumer idempotency patterns
Q86	SQS message tracing with X-Ray
Q87	SQS tag-based cost allocation
Q88	SQS queue policies for S3 event notifications
Q89	SQS to trigger ECS tasks
Q90	Multi-region SQS architecture
Q91	SQS for saga pattern implementation
Q92	SQS for outbox pattern
Q93	SQS consumer graceful shutdown in Go
Q94	SQS with API Gateway direct integration
Q95	SQS producer backpressure patterns
Q96	SQS queue depth alerting runbook
Q97	SQS and circuit breaker pattern
Q98	SQS consumer health check implementation

Q99	SQS interview question: design order processing
Q100	SQS production deployment checklist

Extended Questions (Q78–Q100)

Q78. What is SQS dead-letter queue (DLQ) and redrive policy?

Difficulty: Medium

```
{
  "deadLetterTargetArn": "arn:aws:sqs:us-east-1:123456789:my-queue-dlq",
  "maxReceiveCount": 3
}
```

```
// Message goes to DLQ after maxReceiveCount failed attempts
// DLQ must be same type (standard/FIFO) as source queue

// Create DLQ
dlqURL := createQueue("my-queue-dlq")

// Set redrive policy on source queue
svc.SetQueueAttributes(&sqs.SetQueueAttributesInput{
    QueueUrl: &sourceURL,
    Attributes: map[string]*string{
        "RedrivePolicy": aws.String(`{
            "deadLetterTargetArn": "arn:aws:sqs:us-east-1:123:my-queue-dlq",
            "maxReceiveCount": "3"
        }`),
    },
})

// Monitor DLQ: set CloudWatch alarm on ApproximateNumberOfMessagesVisible
// Alert when DLQ > 0 (messages failing)

// Redrive from DLQ back to source (after fixing bug)
svc.StartMessageMoveTask(&sqs.StartMessageMoveTaskInput{
    SourceArn:           &dlqARN,
    DestinationArn:      &sourceARN,
    MaxNumberOfMessagesPerSecond: aws.Int64(100),
})
```

Q79. What is SQS message filtering with SNS subscription?

Difficulty: Medium

```
// SNS → SQS with filter policies: only deliver matching messages
// Reduces noise, avoids processing irrelevant messages

// SNS subscription filter policy (JSON attribute matching)
filterPolicy := `{
    "event_type": ["order.placed", "order.cancelled"],
    "priority": [{"numeric": [">=", 5]}],
    "region": ["us-east-1", "eu-west-1"]
}`
// Attach to SNS subscription
```

```
svc.SetSubscriptionAttributes(&sns.SetSubscriptionAttributesInput{
    SubscriptionArn: &subARN,
    AttributeName:   aws.String("FilterPolicy"),
    AttributeValue:  aws.String(filterPolicy),
})

// Message attributes must match for delivery
svc.Publish(&sns.PublishInput{
    TopicArn: &topicARN,
    Message:  aws.String(body),
    MessageAttributes: map[string]*sns.MessageAttributeValue{
        "event_type": {DataType: aws.String("String"), StringValue: aws.String("order.placed")},
        "priority":   {DataType: aws.String("Number"), StringValue: aws.String("7")},
        "region":     {DataType: aws.String("String"), StringValue: aws.String("us-east-1")},
    },
})
// Only queues with matching filter receive this message
```

Q80. What is SQS long polling vs short polling?

Difficulty: Easy

```
// Short polling (default, WaitTimeSeconds=0):
// Returns immediately even if no messages
// May return empty responses → wastes API calls + $$$

// Long polling (WaitTimeSeconds=1-20):
// Waits up to N seconds for messages to arrive
// Fewer empty responses, lower cost, lower latency

// Recommended: always use long polling
result, err := svc.ReceiveMessage(&sqs.ReceiveMessageInput{
    QueueUrl:            &queueURL,
    WaitTimeSeconds:     aws.Int64(20),  // wait up to 20s
    MaxNumberOfMessages: aws.Int64(10),
})

// Queue-level default (set once, all consumers benefit)
svc.SetQueueAttributes(&sqs.SetQueueAttributesInput{
    QueueUrl: &queueURL,
    Attributes: map[string]*string{
        "ReceiveMessageWaitTimeSeconds": aws.String("20"),
    },
})

// Cost comparison:
// 1M requests short poll: $0.40
// Same throughput long poll: $0.04 (10x cheaper)
// Plus: reduces empty receives from 90% to <1%
```

Q81. What is SQS message visibility and worker crashes?

Difficulty: Hard

```
// If worker crashes after receiving but before deleting:
// Message becomes visible after VisibilityTimeout → reprocessed
// This is SQS's at-least-once delivery guarantee

// Problem: long-running jobs where VisibilityTimeout < processing time
// Message reappears while still being processed → duplicate processing!
```

```
// Solution: extend visibility timeout during processing
func processWithHeartbeat(ctx context.Context, msg *sqs.Message) error {
    done := make(chan struct{})

    // Heartbeat goroutine: extend visibility every 30s
    go func() {
        ticker := time.NewTicker(30 * time.Second)
        defer ticker.Stop()
        for {
            select {
            case <-done:
                return
            case <-ticker.C:
                svc.ChangeMessageVisibility(&sqs.ChangeMessageVisibilityInput{
                    QueueUrl:          &queueURL,
                    ReceiptHandle:     msg.ReceiptHandle,
                    VisibilityTimeout: aws.Int64(60),  // extend by 60s
                })
            }
        }
    }()

    err := doLongProcessing(ctx, msg)
    close(done)
    return err
}
```

Q82. What is SQS FIFO queue ordering guarantees?

Difficulty: Hard

```
// FIFO: exactly-once processing within message group
// MessageGroupId: ordering scope (like Kafka partition key)
// MessageDeduplicationId: deduplication within 5-minute window

// Send to FIFO queue
svc.SendMessage(&sqs.SendMessageInput{
    QueueUrl:               aws.String("https://sqs.us-east-1.amazonaws.com/123/orders.fifo"),
    MessageBody:            aws.String(orderJSON),
    MessageGroupId:         aws.String(fmt.Sprintf("customer-%d", customerID)),
    MessageDeduplicationId: aws.String(idempotencyKey),
})

// Ordering guarantee:
// SAME MessageGroupId → strict FIFO order
// DIFFERENT MessageGroupId → independent ordering

// Throughput:
// Standard: nearly unlimited TPS
// FIFO: 300 TPS (or 3000 with batching) per queue
// FIFO with high throughput mode: 30,000 TPS

// Deduplication:
// Content-based: SHA-256 of MessageBody (enable on queue)
// Explicit: provide MessageDeduplicationId
// Window: 5 minutes — same ID within 5 min = duplicate (ignored)
```

Q83. What is SQS with Lambda (event source mapping)?

Difficulty: Medium

```
// Lambda: triggered by SQS automatically
// Batch size: 1-10000 messages per invocation
// Lambda polls SQS, manages concurrency automatically

// Lambda function handler
func handler(ctx context.Context, event events.SQSEvent) error {
    var failures []events.SQSBatchItemFailure

    for _, record := range event.Records {
        if err := processRecord(ctx, record); err != nil {
            // Report partial failure (only failed messages go to DLQ)
            failures = append(failures, events.SQSBatchItemFailure{
                ItemIdentifier: record.MessageId,
            })
        }
    }

    if len(failures) > 0 {
        return events.SQSBatchResponse{BatchItemFailures: failures}
    }
    return nil
}

// Event source mapping config
{
    "EventSourceArn": "arn:aws:sqs:...:my-queue",
    "FunctionName": "my-processor",
    "BatchSize": 10,
    "MaximumBatchingWindowInSeconds": 5,
    "FunctionResponseTypes": ["ReportBatchItemFailures"],
    "ScalingConfig": { "MaximumConcurrency": 100 }
}
```

Q84. What is SQS cost optimization?

Difficulty: Medium

```
SQS pricing (us-east-1):
  First 1M requests/month: FREE
  Standard: $0.40 per million requests
  FIFO: $0.50 per million requests
  Data transfer in: free
  Data transfer out: first 100GB free

Cost optimizations:

1. Long polling (biggest saving)
   WaitTimeSeconds=20 → 90% fewer empty receives
   100K messages/day short poll: ~$0.04
   100K messages/day long poll: ~$0.004

2. Batch operations
   SendMessageBatch: up to 10 messages = 1 request (10x savings)
   ReceiveMessage: up to 10 messages = 1 request
   DeleteMessageBatch: up to 10 = 1 request
```

```
3. Message size
   Requests billed per 64KB chunk
   100KB message = 2 requests
   Use SQS Extended Client for messages >256KB (stores in S3)

4. Right-size visibility timeout
   Too short → message reprocessed → duplicate requests
   Too long → failed messages take long to retry

5. Monitor and alarm on DLQ
   Don't let failed messages accumulate (they still cost)
```

Q85. What is SQS vs SNS vs EventBridge comparison?

Difficulty: Medium

```
SQS (Simple Queue Service):
  Pattern: point-to-point queue (producer → consumer)
  Retention: up to 14 days
  Consumers: one consumer group processes each message
  Use: task queues, job processing, decoupling services
  Example: order processing worker reads from order queue

SNS (Simple Notification Service):
  Pattern: pub/sub fan-out (topic → N subscribers)
  Retention: no retention (fire-and-forget)
  Consumers: ALL subscribers receive EVERY message
  Use: notifications, fan-out to multiple queues/lambdas
  Example: order.placed → email queue + analytics queue + inventory queue

EventBridge (formerly CloudWatch Events):
  Pattern: event bus with routing rules
  Retention: can archive and replay events
  Consumers: rules route events to targets (Lambda, SQS, Step Functions)
  Use: complex event routing, cross-account, scheduled events
  Example: route EC2 events to different handlers based on event type

Common patterns:
  SNS → SQS: fan-out to multiple queues (each processes independently)
  EventBridge → SQS: complex routing based on event content
  SQS → Lambda: serverless consumer
```

Q86. What is SQS poison pill message handling?

Difficulty: Hard

```
// Poison pill: message that always fails processing
// Without DLQ: infinite retry loop, blocks queue

// Solution 1: DLQ (primary defense)
// Set maxReceiveCount=3 → after 3 failures → moves to DLQ

// Solution 2: Track receive count in application
func processMessage(msg *sqs.Message) error {
    receiveCount, _ := strconv.Atoi(
        *msg.Attributes["ApproximateReceiveCount"])

    if receiveCount > 3 {
        // Too many retries — log and delete (don't send to DLQ)
        log.Printf("POISON PILL: msg=%s receive_count=%d body=%s",
```

```
            *msg.MessageId, receiveCount, *msg.Body)
        sendAlert(msg)  // alert on-call
        deleteMessage(msg)  // remove from queue
        return nil
    }

    return actualProcessing(msg)
}

// Solution 3: Exponential backoff via ChangeMessageVisibility
func handleWithBackoff(msg *sqs.Message, attempt int) {
    if err := process(msg); err != nil {
        backoff := math.Pow(2, float64(attempt)) * 10  // 10, 20, 40, 80s
        svc.ChangeMessageVisibility(&sqs.ChangeMessageVisibilityInput{
            QueueUrl:          &queueURL,
            ReceiptHandle:     msg.ReceiptHandle,
            VisibilityTimeout: aws.Int64(int64(backoff)),
        })
    }
}
```

Q87. What is SQS server-side encryption?

Difficulty: Easy

```
// SSE (Server-Side Encryption): encrypt messages at rest
// Uses AWS KMS (Key Management Service)

// Enable SSE on queue creation
svc.CreateQueue(&sqs.CreateQueueInput{
    QueueName: aws.String("secure-queue"),
    Attributes: map[string]*string{
        "KmsMasterKeyId":               aws.String("alias/aws/sqs"),  // AWS managed key
        // OR:
        "KmsMasterKeyId":               aws.String("arn:aws:kms:us-east-1:123:key/abc"),  // CMK
        "KmsDataKeyReusePeriodSeconds": aws.String("300"),  // reuse data key for 5 min
    },
})

// SQS SSE uses envelope encryption:
// 1. KMS generates data key (DEK)
// 2. DEK encrypts message
// 3. KMS master key encrypts DEK
// 4. Encrypted DEK stored with message

// Cost: KMS API calls ($0.03/10K)
// KmsDataKeyReusePeriodSeconds: reuse DEK to reduce KMS calls

// In-transit encryption: TLS by default (HTTPS endpoint)
// SQS endpoint: https://sqs.us-east-1.amazonaws.com (always HTTPS)
```

Q88. What is SQS batch processing in Go?

Difficulty: Medium

```
// Process messages efficiently with batching

type SQSWorker struct {
    svc      *sqs.SQS
    queueURL string
```

```
    handler  func(msg *sqs.Message) error
}

func (w *SQSWorker) Run(ctx context.Context) {
    for {
        select {
        case <-ctx.Done():
            return
        default:
            msgs, err := w.receive()
            if err != nil { time.Sleep(time.Second); continue }

            w.processBatch(msgs)
        }
    }
}

func (w *SQSWorker) processBatch(msgs []*sqs.Message) {
    var (
        wg      sync.WaitGroup
        mu      sync.Mutex
        deletes []*sqs.DeleteMessageBatchRequestEntry
    )

    for _, msg := range msgs {
        wg.Add(1)
        go func(m *sqs.Message) {
            defer wg.Done()
            if err := w.handler(m); err != nil {
                log.Printf("failed msg %s: %v", *m.MessageId, err)
                return
            }
            mu.Lock()
            deletes = append(deletes, &sqs.DeleteMessageBatchRequestEntry{
                Id:            m.MessageId,
                ReceiptHandle: m.ReceiptHandle,
            })
            mu.Unlock()
        }(msg)
    }

    wg.Wait()

    // Batch delete successful messages
    if len(deletes) > 0 {
        w.svc.DeleteMessageBatch(&sqs.DeleteMessageBatchInput{
            QueueUrl: &w.queueURL,
            Entries:  deletes,
        })
    }
}
```

Q89. What is SQS queue depth monitoring?

Difficulty: Medium

```
// Key metrics to monitor:
// 1. ApproximateNumberOfMessagesVisible: backlog size
```

```
// 2. ApproximateNumberOfMessagesNotVisible: in-flight (being processed)
// 3. ApproximateAgeOfOldestMessage: how stale is the oldest message?
// 4. NumberOfMessagesSent / NumberOfMessagesDeleted: throughput

// Get queue attributes
result, _ := svc.GetQueueAttributes(&sqs.GetQueueAttributesInput{
    QueueUrl: &queueURL,
    AttributeNames: []*string{
        aws.String("ApproximateNumberOfMessages"),
        aws.String("ApproximateNumberOfMessagesNotVisible"),
        aws.String("ApproximateAgeOfOldestMessage"),
    },
})

visible, _ := strconv.Atoi(*result.Attributes["ApproximateNumberOfMessages"])
inFlight, _ := strconv.Atoi(*result.Attributes["ApproximateNumberOfMessagesNotVisible"])
ageSeconds, _ := strconv.Atoi(*result.Attributes["ApproximateAgeOfOldestMessage"])

// CloudWatch alarms:
// Queue depth > 1000 → scale up consumers
// OldestMessage age > 5 min → consumers stuck/crashed
// DLQ depth > 0 → processing failures (page on-call)

// Auto-scaling consumers:
// Target tracking: keep ApproximateNumberOfMessages / running tasks = N
// Scale out when backlog growing, scale in when draining
```

Q90. What is SQS temporary queues (virtual queues)? Difficulty: Hard

```
// Temporary queues: short-lived queues for request-response pattern
// Without creating thousands of real SQS queues

// SQS Temporary Queue Client (AWS library)
// Creates a "virtual" queue on top of a host queue
// Multiplexes multiple virtual queues over one real queue

// Request-response pattern with SQS
func requestResponse(ctx context.Context, requestQueue string, payload string) (string, error) {
    // Create temporary reply queue
    replyQueueURL := createTempQueue()
    defer deleteTempQueue(replyQueueURL)

    // Send request with reply-to address
    svc.SendMessage(&sqs.SendMessageInput{
        QueueUrl:    &requestQueue,
        MessageBody: aws.String(payload),
        MessageAttributes: map[string]*sqs.MessageAttributeValue{
            "ReplyTo": {
                DataType:    aws.String("String"),
                StringValue: &replyQueueURL,
            },
            "CorrelationId": {
                DataType:    aws.String("String"),
                StringValue: aws.String(uuid.New().String()),
            },
        },
    })
```

```
    // Wait for response
    ctx, cancel := context.WithTimeout(ctx, 30*time.Second)
    defer cancel()
    return waitForResponse(ctx, replyQueueURL)
}
```

Q91. What is SQS access control with IAM?

Difficulty: Medium

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {"AWS": "arn:aws:iam::123456789:role/producer-role"},
      "Action": ["sqs:SendMessage"],
      "Resource": "arn:aws:sqs:us-east-1:123:my-queue"
    },
    {
      "Effect": "Allow",
      "Principal": {"AWS": "arn:aws:iam::123456789:role/consumer-role"},
      "Action": ["sqs:ReceiveMessage", "sqs:DeleteMessage", "sqs:ChangeMessageVisibility"],
      "Resource": "arn:aws:sqs:us-east-1:123:my-queue"
    },
    {
      "Effect": "Allow",
      "Principal": {"Service": "sns.amazonaws.com"},
      "Action": "sqs:SendMessage",
      "Resource": "arn:aws:sqs:us-east-1:123:my-queue",
      "Condition": {
        "ArnEquals": {"aws:SourceArn": "arn:aws:sns:us-east-1:123:my-topic"}
      }
    }
  ]
}
```

Q92. What is SQS message attributes?

Difficulty: Medium

```
// Message attributes: metadata alongside message body
// Up to 10 attributes per message
// Types: String, Number, Binary, String.Array

svc.SendMessage(&sqs.SendMessageInput{
    QueueUrl:    &queueURL,
    MessageBody: aws.String(body),
    MessageAttributes: map[string]*sqs.MessageAttributeValue{
        "EventType": {
            DataType:    aws.String("String"),
            StringValue: aws.String("order.placed"),
        },
        "OrderId": {
            DataType:    aws.String("Number"),
            StringValue: aws.String("42"),
        },
        "TraceId": {
            DataType:    aws.String("String"),
```

```
            StringValue: aws.String(traceID),
        },
    },
})

// Receive with attributes
svc.ReceiveMessage(&sqs.ReceiveMessageInput{
    QueueUrl:              &queueURL,
    MessageAttributeNames: []*string{aws.String("All")},
    AttributeNames:        []*string{aws.String("All")},
})
// msg.MessageAttributes["EventType"].StringValue
// msg.Attributes["ApproximateReceiveCount"]
```

Q93. What is SQS delay queues?

Difficulty: Medium

```
// Delay queue: message invisible for N seconds after send
// Use: schedule future processing, rate limiting, retry with delay

// Queue-level delay (all messages delayed)
svc.CreateQueue(&sqs.CreateQueueInput{
    QueueName: aws.String("delayed-queue"),
    Attributes: map[string]*string{
        "DelaySeconds": aws.String("900"),  // 15 min delay
    },
})

// Per-message delay (overrides queue default)
svc.SendMessage(&sqs.SendMessageInput{
    QueueUrl:     &queueURL,
    MessageBody:  aws.String(body),
    DelaySeconds: aws.Int64(300),  // 5 min delay
})

// Max delay: 900 seconds (15 minutes)
// Use case: send "order reminder" email 1 hour after order
// Use case: retry failed webhook after exponential backoff
// Use case: deferred payment processing

// Note: FIFO queues don't support per-message delay
// Only queue-level delay for FIFO
```

Q94. What is SQS vs Kafka: when to use which?

Difficulty: Medium

SQS:
- Simple setup (fully managed, no ops)
- Auto-scaling consumers
- Pay per use (no idle cost)
- Native AWS integrations (Lambda, SNS, EventBridge)
- At-least-once delivery built-in (DLQ)
- No message replay (once consumed + deleted = gone)
- 256KB message size limit
- No consumer offset tracking
- Max retention: 14 days

Use: task queues, job processing, simple async decoupling

```
Kafka:
  ■ Message replay (consumers can re-read history)
  ■ Very high throughput (millions/sec)
  ■ Long retention (unlimited with tiered storage)
  ■ Multiple independent consumer groups
  ■ Stream processing (Kafka Streams, Flink)
  ■ Operational complexity (or pay for Confluent/MSK)
  ■ No automatic scaling (manage partitions)
  ■ Complex consumer offset management
  Use: event sourcing, audit logs, stream processing, analytics

Decision:
  < 1M msg/day, AWS-native, simple consumers → SQS
  Event replay needed, analytics, high throughput → Kafka
  Mixed: use SQS for tasks, Kafka for events
```

Q95. What is SQS extended client for large messages?

Difficulty: Medium

```
// SQS max message size: 256KB
// Extended client: store large payloads in S3, pointer in SQS

import "github.com/awsdocs/amazon-sqs-developer-guide"

// All messages → S3
extClient := sqsextendedclient.New(svc, s3Svc, "my-bucket",
    sqsextendedclient.AlwaysUseS3(true))

// Only messages > threshold → S3
extClient := sqsextendedclient.New(svc, s3Svc, "my-bucket",
    sqsextendedclient.PayloadSizeThreshold(100*1024))  // 100KB threshold

// Send (auto-uploads to S3 if needed)
extClient.SendMessage(&sqs.SendMessageInput{
    QueueUrl:    &queueURL,
    MessageBody: aws.String(largePayload),
})

// Receive (auto-downloads from S3)
result, _ := extClient.ReceiveMessage(&sqs.ReceiveMessageInput{
    QueueUrl: &queueURL,
})
// msg.Body contains original large payload (downloaded from S3)

// Cleanup: extended client deletes S3 object when SQS message deleted
extClient.DeleteMessage(&sqs.DeleteMessageInput{
    QueueUrl:      &queueURL,
    ReceiptHandle: msg.ReceiptHandle,
})
```

Q96. What is SQS cross-account access?

Difficulty: Medium

```
// Account A: producer
// Account B: queue owner

// Step 1: Account B adds resource policy to allow Account A
```

```
policy := `{
    "Statement": [{
        "Effect": "Allow",
        "Principal": {"AWS": "arn:aws:iam::ACCOUNT-A:root"},
        "Action": "sqs:SendMessage",
        "Resource": "arn:aws:sqs:us-east-1:ACCOUNT-B:shared-queue"
    }]
}`

// Step 2: Account A assumes role or uses direct access
// Producer in Account A:
sess, _ := session.NewSession(&aws.Config{
    Region: aws.String("us-east-1"),
})
svc := sqs.New(sess)
svc.SendMessage(&sqs.SendMessageInput{
    QueueUrl:    aws.String("https://sqs.us-east-1.amazonaws.com/ACCOUNT-B/shared-queue"),
    MessageBody: aws.String(body),
})

// Cross-account patterns:
// Org-wide event bus: central SQS queue receives from all accounts
// SNS cross-account: topic in one account, SQS subscription in another
```

Q97. What is SQS approximate vs exact message counts?

Difficulty: Medium

```
SQS counts are APPROXIMATE:
  ApproximateNumberOfMessages: may lag by seconds
  ApproximateNumberOfMessagesNotVisible: may be inaccurate
  ApproximateAgeOfOldestMessage: approximate

Why approximate?
  SQS is distributed across multiple servers
  Counting is eventually consistent
  Trade-off: high availability over exact consistency

Implications:
  Don't use for billing/exact counts
  Use for: autoscaling decisions (approximate is fine)
  Don't rely on count=0 for "queue is empty" (use polling)

Getting better accuracy:
  Purge + wait 60s → count more accurate
  Use CloudWatch metrics (15-min resolution)
  Count in your application code (track sent/processed)

FIFO queue counts:
  More accurate because FIFO queues don't use same distributed model
  But still marked as "approximate" in docs
```

Q98. What is SQS with AWS CDK/Terraform?

Difficulty: Medium

```
// AWS CDK (TypeScript)
import * as sqs from 'aws-cdk-lib/aws-sqs';

const dlq = new sqs.Queue(this, 'OrdersDLQ', {
```

```
  retentionPeriod: Duration.days(14),
});

const queue = new sqs.Queue(this, 'OrdersQueue', {
  visibilityTimeout: Duration.seconds(300),
  retentionMessagePeriod: Duration.days(4),
  deadLetterQueue: {
    queue: dlq,
    maxReceiveCount: 3,
  },
  encryption: sqs.QueueEncryption.KMS_MANAGED,
});
```

```
# Terraform
resource "aws_sqs_queue" "orders_dlq" {
  name                      = "orders-dlq"
  message_retention_seconds = 1209600  # 14 days
}

resource "aws_sqs_queue" "orders" {
  name                       = "orders"
  visibility_timeout_seconds = 300
  message_retention_seconds  = 345600  # 4 days
  redrive_policy = jsonencode({
    deadLetterTargetArn = aws_sqs_queue.orders_dlq.arn
    maxReceiveCount     = 3
  })
  kms_master_key_id = "alias/aws/sqs"
}
```

Q99. What is SQS message deduplication strategies?

Difficulty: Medium

```
// Problem: SQS is at-least-once → consumer may receive duplicates
// Solution: idempotent consumers

// Strategy 1: Database unique constraint
func processOrder(ctx context.Context, msg *sqs.Message) error {
    var order Order
    json.Unmarshal([]byte(*msg.Body), &order)

    _, err := db.ExecContext(ctx,
        "INSERT INTO processed_messages (message_id) VALUES ($1) ON CONFLICT DO NOTHING",
        *msg.MessageId,
    )
    if err != nil { return err }

    // Check if already processed (0 rows inserted = duplicate)
    // ... or use INSERT ... ON CONFLICT DO NOTHING RETURNING id
    return processOrderBusiness(ctx, order)
}

// Strategy 2: Redis SET NX
func isProcessed(ctx context.Context, messageID string) bool {
    result, _ := rdb.SetNX(ctx, "processed:"+messageID, "1", 24*time.Hour).Result()
    return !result  // false = not processed (we set it), true = already exists
}
```

```
// Strategy 3: FIFO queue with MessageDeduplicationId
// SQS itself deduplicates (5-minute window)
// Best for: preventing duplicates at source

// Strategy 4: Outbox + message ID
// Producer writes message_id to outbox → consumer checks processed table
```

Q100. What is SQS production deployment checklist?

Difficulty: Medium

Before go-live with SQS in production:

Queue configuration:
- ■ Appropriate VisibilityTimeout (> max processing time)
- ■ DLQ configured with maxReceiveCount=3-5
- ■ Message retention period set (default 4 days)
- ■ Long polling enabled (WaitTimeSeconds=20)
- ■ SSE enabled (KmsMasterKeyId set)
- ■ Resource policy: least privilege

Consumer:
- ■ Idempotent message processing
- ■ Batch processing (up to 10 messages)
- ■ Heartbeat for long-running jobs (ChangeMessageVisibility)
- ■ Graceful shutdown (finish in-flight before exit)
- ■ Error handling (don't delete on failure)
- ■ Structured logging with message IDs

Monitoring:
- ■ CloudWatch alarm: ApproximateAgeOfOldestMessage > threshold
- ■ CloudWatch alarm: DLQ depth > 0
- ■ CloudWatch alarm: consumer error rate
- ■ Queue depth dashboard for capacity planning

Operations:
- ■ DLQ redrive procedure documented
- ■ Runbook for consumer outage
- ■ Autoscaling policy based on queue depth
- ■ IAM roles (not access keys) for authentication